



GaiaCom

Руководство пользователя, оператора и
разработчика

Sovereign Beta v2 · Система и архитектура · Сентябрь 2026

Post-Quantum Communication Infrastructure · Zero Server Trust · Federated by Design

GaiaCom - Руководство пользователя, оператора и разработчика

Статус документации: Sovereign Beta v2 / снимок репозитория, сентябрь 2026 г.

Продукт: GaiaCom

Тип документа: Руководство пользователя · Руководство оператора узла · Руководство по развертыванию · Справочник разработчика

Основа: Исходный код и документация предоставленного снимка `GaiaCom-main`, а также отдельно предоставленная подробная карта архитектуры.

Важно: Это руководство описывает фактически предоставленное состояние репозитория. Там, где исходный код, README, более старые документы и карта архитектуры расходятся, это отмечено явно. В русской редакции отдельно предоставленная карта архитектуры включена в **Приложение А в полном переводе на русский язык**.

Содержание

1. О руководстве
2. GaiaCom в одном предложении
3. Статус продукта и границы релиза
4. Источник истины и известные расхождения документации
5. Модель безопасности и границы доверия
6. Основные термины

Часть I - Руководство пользователя

1. Первый запуск, регистрация и онбординг
2. Вход, локальная блокировка и восстановление
3. Подключение нового устройства
4. Навигация и Dashboard
5. Идентичность, GaiaID, контакты и Trust Passport
6. Quantum Chat / Direct Messaging
7. GaiaMail
8. SMTP Legacy Bridge
9. Группы, комнаты и подканалы
10. Public Channels
11. GSN - GaiaSocialNetwork
12. GaiaDrive
13. GaiaDrop
14. Network Health
15. Security Center / GaiaShield
16. Abuse Center, Governance и Gaia's Eyes
17. Профиль и персональные настройки
18. Горячие клавиши

Часть II - Руководство оператора узла и развертывания

1. Модель эксплуатации
2. Предварительные требования
3. Локальная среда разработки
4. Production-сборка
5. Подготовка Linux-сервера
6. Production-конфигурация с `gaiacom setup`
7. `gaiacom doctor` и `gaiacom version`
8. Bootstrap Governance
9. Усиление systemd

10. NGINX, TLS и Reverse Proxy
11. Firewall и сетевая экспозиция
12. Node Registry и контролируемая федерация
13. Хранилище: локальный Object Store и S3/MinIO
14. Production-настройка SMTP
15. Health Checks, Metrics и Logs
16. Резервное копирование и восстановление
17. Обновления и Rollback
18. Production-чек-лист

Часть III - Руководство разработчика

1. Структура репозитория
2. Архитектура Frontend
3. Архитектура Backend
4. Персистентность и SQLite
5. Sovereign Crypto Core, WASM и Desktop
6. Android-платформа
7. Обзор API
8. Security- и Quality-Gates
9. Безопасная разработка и расширение

Часть IV - Эксплуатационная помощь

1. Troubleshooting
2. Известные ограничения Sovereign Beta v2
3. Глоссарий
4. Сопровождение документации

Приложение

- A. Полная карта архитектуры
-

1. О руководстве

GaiaCom - это не одно веб-приложение. Предоставленное состояние включает интерфейс React/Vite, Backend на Go, персистентность SQLite, основанный на Rust слой Sovereign Crypto, WebAssembly, Desktop-host на Tauri, модульную Android-платформу, файлы развертывания NGINX/systemd, логику федерации, подсистемы Governance и Security, а также наборы adversarial-тестов.

Поэтому руководство предназначено для четырех групп читателей:

- **Конечные пользователи**, работающие с GaiaCom.
- **Администраторы и операторы узлов**, устанавливающие, защищающие и эксплуатирующие узел GaiaCom.
- **Разработчики**, которым необходимо понимать или расширять репозиторий.
- **Security-reviewer'ы**, оценивающие границы безопасности, тестовые gates и известные ограничения.

Карта архитектуры в Приложении А служит углубленным техническим справочником.

Основная часть руководства сосредоточена на **использовании, настройке, эксплуатации и практических рабочих процессах**.

2. GaiaCom в одном предложении

GaiaCom - это **федеративная коммуникационная платформа с клиентским сквозным шифрованием и гибридной криптографией, ориентированной на постквантовую устойчивость**, объединяющая Direct Messaging, почту, групповые комнаты, публичные каналы, социальные функции, зашифрованное файловое хранилище, анонимные или псевдонимные текстовые обращения, Governance и федерацию узлов в одной платформе.

Центральный принцип проектирования - **Zero Server Trust**: корректно эксплуатируемому серверу GaiaCom не должен требоваться конфиденциальный пользовательский контент в открытом виде, чтобы обеспечивать транспорт, доставку, федерацию и хранение.

3. Статус продукта и границы релиза

Предоставленный снимок следует рассматривать как **Sovereign Beta v2**. Не каждый элемент UI или экспериментальный компонент, видимый в исходном коде, автоматически считается разрешенной production-функцией.

3.1 Активно или предусмотрено в Beta-состоянии

Область	Статус в предоставленном состоянии
Native E2EE Direct Messages	Активно
Групповые комнаты и подканалы	Активно
GaiaMail	Активно
SMTP Bridge	Активно, но является явным Security Downgrade
GaiaDrive	Активно
Текстовые обращения GaiaDrop	Активно
GaiaProof	Активно
TrustMesh	Активно, локально / policy-based
Trust Passport / Gaia Passport	Активно
Предупреждения о смене ключа	Активно
Public Channels	Активно
GSN Social Layer	Активно
Controlled Federation	Активно, известные/одобренные узлы
Node Registry MVP	Активно
Governance / Reviewer	Активно
GaiaShield / Security Center	Активно
Sovereign Accelerated	Активно
Sovereign Top Secret	Активно только при выполнении требуемых capabilities
Browser HQC Provider	Активно через изолированный Worker/WASM
Tauri Native Crypto Bridge	Активно
S3/MinIO Object Store Adapter	Опционально активно
Модульная база Android	Technical Beta

3.2 Пока намеренно не считать полностью выпущенными

Release-документация исключает из статуса полностью готовых, среди прочего, следующие области:

- Файловые, медиа- и графические вложения в чате до завершения Attachment Audit.
- Открытую публичную федерацию без контролируемого допуска.

- Полностью открытую регистрацию узлов.
- Глобально синхронизированный Abuse Consensus.
- Полный Mobile Sovereign v1 messaging path.
- Triple Ratchet или ratchet-based Forward/Post-Compromise Secrecy.
- Identity-signed per-device Sovereign Prekeys как полностью завершённый путь.
- GaiaDrop с вложениями.
- Полностью автоматизированную Update Pipeline.

Практическое правило: если UI уже показывает кнопку экспериментальной функции, но Release Matrix все еще помечает ее как disabled или unaudited, приоритет имеет **граница релиза**.

4. Источник истины и известные расхождения документации

При подготовке руководства README, исходный код, deployment-файлы и отдельно предоставленная карта архитектуры были сопоставлены между собой. Выявлено несколько различий, имеющих значение для установки и безопасности.

4.1 Хеширование паролей: Argon2id

GaiaCom использует **Argon2id** как обязательный стандарт серверного хеширования паролей. Регистрация, вход, смена пароля и удаление учетной записи должны последовательно использовать единый центральный путь Argon2id.

Для новых и мигрированных учетных записей Argon2id является целевым и релизным стандартом. Любые оставшиеся legacy-хеши должны контролируемо мигрироваться в ходе текущего перехода; новые хеши паролей создаются исключительно с Argon2id. Поэтому карта архитектуры в Приложении А соответствует целевому криптографическому состоянию.

4.2 Файл конфигурации Vite

В карте архитектуры местами указан `vite.config.js`. В предоставленном репозитории файл называется:

```
Frontend/frontend/vite.config.mjs
```

4.3 Лицензия: GNU AGPLv3

GaiaCom последовательно публикуется по лицензии **GNU Affero General Public License v3 (AGPLv3 / AGPL-3.0-only)**. Поэтому `LICENSE`, README, метаданные пакетов и уведомления в исходном коде должны единообразно указывать AGPLv3.

4.4 Не считать `OPEN_REGISTRATION=false` надежным Security Gate регистрации пользователей

`gaiacom setup` записывает `OPEN_REGISTRATION=false` в production-профиль, если значение отсутствует. Однако в проверенном пути регистрации Auth эта переменная не просматривается как однозначный авторитетный server-side gatekeeper для `RegisterUser`.

Рекомендация для эксплуатации: если публичная регистрация пользователей должна быть запрещена, не полагайтесь только на эту переменную. Требуемый gate следует явно реализовать в Auth-пути и протестировать.

4.5 Локальные порты Frontend/Backend

Frontend-скрипт по умолчанию запускает Vite так:

```
vite --host 127.0.0.1
```

Vite обычно использует порт `5173`. Dev-CORS-путь Backend в текущем снимке, напротив, ориентирован на локальные Origins вроде `http://localhost:3000` и `http://localhost:1420`.

Поэтому для воспроизводимой разработки в этом руководстве явно используется порт **3000**.

4.6 Desktop API закреплен на `beta.gaiacom.de`

Frontend API Client в Desktop-режиме содержит:

```
https://beta.gaiacom.de
```

CSP Tauri также ориентирован на этот адрес. Для Desktop-сборки под собственный узел необходимо **осознанно изменить API Origin и CSP и пересобрать клиент**.

4.7 Production Provisioning: CLI имеет приоритет над UI Secret Generator

В ряде более старых Operator-документов создание Node Secrets описывается через раздел Security Center / Nodesystem. Актуальный production-путь репозитория предусматривает `gaiacom setup`. Он атомарно создает secrets, формирует окружение с ограничительными правами и затем может быть проверен через `gaiacom doctor`.

В этом руководстве `gaiacom setup` считается каноническим production-путем.

4.8 Entrypoint сборки Backend

Для исполняемого сервера фактический Entrypoint:

```
Backend/cmd/gaiacom
```

Поэтому production-build следует выполнять через `./cmd/gaiacom`, а не универсальной сборкой корня Backend package.

5. Модель безопасности и границы доверия

5.1 Что GaiaCom должен защищать

Система спроектирована для поддержки следующих целей защиты:

- Конфиденциальность native-сообщений и файлов за счет client-side encryption.
- Аутентичность и целостность криптографических Envelopes.
- Устойчивость к Downgrade- и Key-Substitution-атакам.
- Подписанную и устойчивую к Replay server-to-server коммуникацию.
- Минимизацию открытого текста на сервере.
- Ограничение привилегий операторов и reviewer'ов.
- Прослеживаемые Security Events через GaiaShield и Audit Chains.
- Криптографически защищенные Trust/Disclosure Workflows.

5.2 Что GaiaCom явно не решает автоматически

Zero Server Trust не означает "Zero Risk".

Скомпрометированное или разблокированное конечное устройство может раскрыть открытый текст, session material или пользовательский ввод. Скомпрометированная операционная система может обеспечивать keylogging, screen capture или доступ к памяти. Вариант Web/PWA дополнительно опирается на то, что JavaScript-приложение, в данный момент выдаваемое узлом, не было злонамеренно заменено.

Для high-assurance сценариев **подписанный, воспроизводимо собранный native-клиент** концептуально сильнее веб-клиента, который при каждом посещении заново загружается с узла.

GaiaCom также **не является сетью анонимизации**. Метаданные федерации, доставки, времени, учетных записей и сети могут оставаться чувствительными даже при зашифрованном содержимом сообщений.

5.3 Native-коммуникация и SMTP

Native-коммуникация GaiaCom остается внутри криптографии GaiaCom и модели федерации GaiaCom.

При использовании SMTP сообщение покидает эту границу доверия. Поэтому Bridge должен восприниматься и в UI, и при эксплуатации как **осознанный Downgrade**. После передачи традиционной почтовой инфраструктуре действуют риски транспорта, провайдера и метаданных этой инфраструктуры.

6. Основные термины

GaiaID

Федеративный адрес пользователя вида:

```
@username:node.example.org
```

В частности, Node-часть определяет, куда должна маршрутизироваться федеративная коммуникация.

Sovereign Accelerated

Более производительный Sovereign-режим. В текущем дизайне он объединяет hybrid KEM derivation с двухступенчатой каскадной аутентифицированной шифрацией.

Sovereign Top Secret

Более высокий криптографический уровень с дополнительным каскадом шифров и дополнительными требованиями к подписи/capabilities. Он может включаться только тогда, когда обе стороны связи и путь сообщения доказуемо поддерживают требуемые capabilities.

GaiaProof

Формат доказательства, позволяющий криптографически подтверждать релевантные коммуникационные или abuse-события без введения универсального административного ключа расшифрования.

TrustMesh

Локальная, связанная с федерацией система репутации и friction. Это не глобальный "индекс истины".

Gaia Passport / Trust Passport

Представление сведений об идентичности, Human Proof и доверии, связанных с GaiaID.

Gaia's Eyes

Контролируемый Disclosure Workflow для доказательств, добровольно либо в соответствии с policy предоставляемых в Governance/Reviewer-процессах.

GaiaShield

Security-подсистема для аномалий, Audit Events, защитного Middleware, Redaction и видимости для операторов.

Часть I - Руководство пользователя

7. Первый запуск, регистрация и онбординг

7.1 Стартовый экран

Область аутентификации предлагает несколько путей входа:

- Login
- Registration
- Recovery
- Device Pairing
- GaiaDrop для внешних текстовых обращений без обычного входа в аккаунт

7.2 Регистрация

Для новой учетной записи UI требует как минимум имя пользователя, пароль и необходимые юридические и/или Beta-подтверждения.

Текущее состояние Backend применяет server-side ограничения правдоподобия к именам пользователей и паролям. Для пароля предусмотрена минимальная длина **12 символов**.

В ходе регистрации криптографический материал идентичности создается **на стороне клиента**. Сюда относится Recovery/Mnemonic-материал, которым сервер не должен располагать в виде открытой резервной копии.

7.3 SetupWizard - пять основных фаз

Фаза 1 - Условия и Beta-уведомления

Пользователь подтверждает, среди прочего:

- Условия прочитаны.
- Beta-статус понятен.
- Постоянная доступность не гарантируется.
- SMTP находится вне полной native-модели безопасности.
- Ответственность за хранение Recovery/Credentials принимается пользователем.
- Использование является законным.

Фаза 2 - Сохранение Recovery

Recovery Phrase отображается локально. Пользователь должен подтвердить, что сохранил ее.

Рекомендация: никогда не храните Recovery-материал в чате, электронной почте, cloud-заметках или скриншотах, если конкретное место хранения не было осознанно выбрано как безопасное Recovery-хранилище.

Фаза 3 - Идентичность и GaiaID

Пользователь создает или регистрирует идентичность на узле. В результате формируется федеративный GaiaID.

Пример:

```
@rene:node.example.org
```

В зависимости от состояния UI могут предлагаться доступные узлы, собственный узел пользователя или fallback-узлы.

Фаза 4 - Введение в безопасность

Онбординг объясняет основные уровни:

- Идентичность
- Шифрование сообщений
- Сеть/федерация
- Прозрачность и Trust

Фаза 5 - Начало работы

Типичные быстрые переходы:

- Inbox
- Contacts
- Channels
- Security Center

7.4 First-Run Onboarding

Помимо SetupWizard, предусмотрен First-Run Onboarding для профиля, аватара, Security-уведомлений, подтверждения Backup и первого перехода пользователя в приложение.

8. Вход, локальная блокировка и восстановление

8.1 Вход

Обычный Login аутентифицирует пользователя в Backend и инициализирует локальное состояние клиента. Access- и Refresh-Tokens обрабатываются отдельно от собственно Recovery-материала.

8.2 Локальная блокировка сессии

GaiaCom может локально заблокировать уже аутентифицированный интерфейс после периода бездействия. В зависимости от активной конфигурации разблокировка выполняется через:

- Пароль
- PIN
- WebAuthn PRF / аппаратно привязанную derivation

Локальная блокировка не равна полному Logout.

8.3 Logout

Logout завершает текущую серверную сессию либо соответствующий Token Path и очищает локальное состояние аутентификации.

8.4 Импорт Recovery-файла

Раздел Recovery может импортировать зашифрованные GaiaCom Recovery-файлы, например формата `.json` или `.gaiacom-recovery.json`.

Типовой процесс:

1. Выбрать Recovery-файл.
2. Ввести пароль Recovery-файла.
3. Установить новый локальный пароль разблокировки.
4. Расшифровать Recovery-данные.
5. Локально восстановить Mnemonic/Identity-материал.
6. Заново инициализировать состояние Account/Identity.

Важно: безопасность Recovery-файла определяется безопасностью его пароля и места хранения.

9. Подключение нового устройства

GaiaCom отделяет **Device Pairing** от обычной Browser/Login-сессии.

9.1 На новом устройстве

В области Pairing:

1. Ввести данные учетной записи.
2. Назначить имя или Label устройства.
3. Запросить Pairing.
4. GaiaCom генерирует одноразовый Pairing Code или URI вида:

```
gaiacom://pairing/<pairing-id>/<one-time-secret>
```

Его можно передать в виде кода/QR уже доверенному устройству.

9.2 На уже доверенном устройстве

В профиле, раздел **Devices**:

1. Вставить или отсканировать Pairing Code.
2. Проверить запрос Pairing.
3. Выбрать активную идентичность.
4. Recovery/Sovereign-материал криптографически упаковывается для нового устройства.
5. Подписать разрешение.
6. Подтвердить Pairing.

9.3 Завершение Pairing на новом устройстве

Новое устройство проверяет наличие разрешения, consume'ит Pairing и сохраняет разрешенный локальный ключевой материал в защищенном виде.

9.4 Последующее управление устройствами

В зависимости от состояния Profile позволяет:

- отзывать активные Sessions,
- отзывать зарегистрированные Crypto Devices,
- ротировать текущие Device Keys,
- одобрять новые Pairings.

При потере устройства следует проверить и при необходимости отозвать как **Sessions**, так и **Device Keys**.

10. Навигация и Dashboard

Основная навигация разделена на три крупных области.

10.1 Workspace

- Dashboard
- GaiaMail
- SMTP Mail
- Quantum Chat
- Group Chats

GaiaMail и SMTP предоставляют папки Mailbox, в том числе:

- Inbox
- Drafts
- Sent
- Starred
- Important
- Snoozed
- Archive
- Spam
- Trash

10.2 Network

- Public Channels
- GSN Social
- Network Health
- Security Center
- Abuse Center

10.3 Personal

- Address Book / Contacts
- My Profile
- GaiaDrive
- GaiaDrop

В нижней части навигации, в зависимости от клиента, находятся элементы управления Theme, Language, Lock, Logout, статусом Quantum Shield и информацией о версии.

10.4 Dashboard

Dashboard - командный центр. Среди прочего он показывает:

- статус Session/Security,
 - статус Network/Uplink,
 - активную идентичность,
 - криптографические Status Indicators,
 - подключенные комнаты,
 - текущую или новую Mail/Chat-активность,
 - Quick Actions,
 - уведомление об установке PWA, если применимо.
-

11. Идентичность, GaiaID, контакты и Trust Passport

11.1 Добавление контактов

Контакты можно искать или добавлять по GaiaID.

Пример:

```
@alice:node.example.org
```

GaiaID формально валидируется до использования в качестве федеративного адреса.

11.2 Профиль контакта

Профиль контакта объединяет:

- GaiaID
- Profile Information
- Trust/Passport Data
- Human Proof Status
- Key/Fingerprint Information
- Key History

11.3 Проверка Fingerprint

Для особо чувствительных контактов отображаемый Fingerprint следует сверять **через второй, уже доверенный канал**.

11.4 Key Change Warning

GaiaCom ведет локальную, ориентированную на append-only Key History. Если ожидаемый ключ получателя изменился, клиент может выдать предупреждение.

Предупреждение не означает автоматически атаку. Возможные легитимные причины:

- новое устройство,
- ротация ключей,
- восстановление аккаунта,
- переустановка.

Для важных контактов изменение все равно следует проверить.

11.5 Human Proof

Workflow Human Proof создает или обновляет криптографическое доказательство человеческого взаимодействия. Это часть Passport/Trust-системы, а не универсальная юридическая проверка личности.

12. Quantum Chat / Direct Messaging

12.1 Открытие чата

Откройте Quantum Chat и выберите контакт. Header может показывать идентичность, Presence, Trust/Role Information и E2EE/Quantum Status.

12.2 Отправка сообщения

Native-сообщения чата до Upload преобразуются на стороне клиента в зашифрованный Envelope. Backend маршрутизирует или сохраняет данные Envelope, не нуждаясь в расшифровании содержимого сообщения для обычного пути доставки.

12.3 Accelerated и Top Secret

Accelerated

Подходит как стандартный Sovereign-режим.

Top Secret

Более сильный каскад с более строгими требованиями к capabilities.

Клиент обязан работать по принципу **fail closed**: если отсутствуют требуемые primitives, ключи получателя или capability proofs, он не должен незаметно переключаться на более слабый вариант.

12.4 Presence и состояние набора текста

Чат поддерживает Heartbeat/Presence и индикаторы набора текста. Это функции метаданных; их не следует путать с шифрованием содержимого.

12.5 Действия с сообщением

В зависимости от сообщения и прав доступны:

- Ответ
- Редактирование
- Удаление
- Reactions
- Локальное сохранение
- Локальное закрепление
- Статус прочтения

12.6 Поиск в чате

Интерфейс содержит функции поиска по текущей беседе. Если поиск выполняется локально по расшифрованным данным, его не следует ошибочно считать серверным поиском по открытому тексту.

12.7 Блокировка или жалоба на контакт

Контакт можно заблокировать в чате либо отправить жалобу через Abuse Workflow.

12.8 GaiaProof / Export

Для релевантных коммуникационных историй могут быть доступны функции GaiaProof/Export. Зашифрованные экспорты следует защищать отдельным сильным паролем экспорта.

12.9 Вложения и Voice Notes

Исходный код содержит UI- и development-компоненты для файлов, GaiaDrive Sharing и Voice Notes.

Граница релиза: согласно контракту Sovereign Beta v2 файловые/медийные вложения в чате пока не следует считать полностью выпущенными. Для production-документации они остаются экспериментальными до завершения запланированного аудита и обновления Release Matrix.

13. GaiaMail

GaiaMail реализует mailbox-oriented модель коммуникации внутри приложения GaiaCom.

13.1 Создание нового письма

Откройте Composer и используйте native GaiaCom Mode.

Типичные поля:

- Получатель
- Тема
- Текст сообщения
- Форматирование: жирный, курсив, подчеркивание, заголовок, код

Для native-получателей интерфейс использует GaiaID/контактную информацию.

13.2 Черновики

Composer поддерживает управление Drafts и автоматическое либо периодическое сохранение состояния черновика.

13.3 Отложенная отправка

Backend и Composer содержат Scheduler Path для отправки по расписанию.

Для запланированных сообщений узел должен быть работоспособен в соответствующий момент и иметь возможность обработать запланированную доставку.

13.4 Чтение сообщения

В зависимости от типа сообщения Reader предоставляет, среди прочего:

- Ответ
- Ответ всем
- Пересылку
- Архивирование
- Удаление
- Spam
- Snooze
- Labels
- GaiaProof/Disclosure-функции
- Профиль отправителя/получателя

13.5 Предупреждающие баннеры

Reader может отображать предупреждения о:

- Spam/Quarantine
- SMTP Legacy/Downgrade
- измененных ключах контакта

13.6 Правила фильтрации

Mailbox Rules могут применять действия по критериям вроде отправителя или темы, например:

- Important
- Star
- Read
- Label

13.7 Подпись и настройки Mailbox

Profile содержит настройки Mailbox, включая подпись, часть Locale-настроек и keyboard-oriented параметры управления.

14. SMTP Legacy Bridge

SMTP Bridge обеспечивает совместимость с традиционной электронной почтой.

14.1 Значение для безопасности

SMTP **не является native GaiaCom E2EE-путем**. Как только сообщение передается Legacy Gateway, GaiaCom не может обеспечивать свои гарантии безопасности за пределами собственной инфраструктуры.

UI должен заметно маркировать этот переход как Downgrade.

14.2 Создание SMTP-сообщения

В Composer выберите SMTP/Legacy. После этого разрешены внешние адреса электронной почты.

Перед отправкой пользователь должен осознанно подтвердить Downgrade Warning.

14.3 Обязанности оператора

Production-настройка SMTP дополнительно требует:

- аутентифицированный SMTP Relay,
- обязательный TLS,
- корректный Sender Domain,
- SPF,
- DKIM,
- DMARC с содержательной Enforcement Policy,
- защиту от Header/CRLF Injection,
- мониторинг Abuse/Rate Limits.

15. Группы, комнаты и подканалы

15.1 Создание комнаты

Новую комнату можно создать через **Group Chats**.

Возможные свойства:

- Name
- Description
- Avatar
- Private Room
- Crisis Room
- Top Secret

15.2 Приватная комната

Приватная комната не должна обнаруживаться так же, как обычная публичная. Вход возможен по приглашению или Join Request.

15.3 Подканалы

Внутри комнаты можно создавать Channels для разделения тематик.

15.4 Slow Mode

Групповая область поддерживает интервалы Slow Mode, включая:

0 / 5 / 10 / 30 / 60 / 300 seconds

15.5 Участники и роли

В зависимости от прав:

- Member
- Admin
- Owner

Операторы комнаты могут искать участников, менять роли, удалять участников и передавать Ownership.

15.6 Join Requests

Для приватных комнат запросы на вступление можно принять или отклонить.

15.7 Invitation Links

Приглашения можно ограничивать по времени или числу использований, например:

- 1 час
- 1 день
- 7 дней
- без ограничения

и:

- 1 использование
- 5 использований
- 10 использований
- без ограничения

15.8 Moderation Logs

Административные/модерационные действия, связанные с комнатой, записываются для предусмотренной аудитуемости.

16. Public Channels

Public Channels - это публичные Broadcast/Discussion-области.

16.1 Поиск и подписка на каналы

Пользователи могут обнаруживать и искать каналы, подписываться на них и отписываться.

16.2 Posts

В зависимости от прав:

- создать Post,
- редактировать Post,
- удалить Post,
- закрепить Post,
- добавить Reaction.

16.3 Комментарии

Каналы могут разрешать или запрещать комментарии. Модераторы могут управлять комментариями.

16.4 Модерация и Reports

На каналы или контент можно пожаловаться. Категории системы включают, например:

- Spam
- Phishing
- Malware
- Harassment
- Illegal Content
- Threat
- Other

Report - это Governance Signal, а не автоматическое доказательство.

17. GSN - GaiaSocialNetwork

GSN - децентрализованный Social Layer GaiaCom.

17.1 Feeds

В зависимости от представления:

- Node/General Feed
- Following Feed

17.2 Posts

Пользователи могут:

- создавать Posts,
- удалять собственные Posts,
- реагировать,
- комментировать.

17.3 Profiles и Follows

В зависимости от настроек GSN Profiles содержат поля вроде:

- Display Name
- Bio
- Website
- Avatar

На другие Profiles можно подписываться и отписываться.

17.4 Жалобы

Profiles и Posts могут быть переданы в Abuse/Governance Path через Report.

17.5 Связь с другими областями GaiaCom

UI связывает Social Profiles с Chat, Rooms и Public Channels, позволяя переходить между публичной и приватной коммуникацией.

18. GaiaDrive

GaiaDrive - актуальная область зашифрованного персонального хранилища. Старый путь `VaultPane/useVault` следует считать Legacy-кодом.

18.1 Разблокировка Drive

Пользователь локально разблокирует GaiaDrive с помощью Drive Password. Этот пароль используется для локальной Key Derivation и не должен рассматриваться как слабый Convenience Password.

18.2 Заметки

GaiaDrive может управлять зашифрованными локальными заметками.

Возможные действия:

- создать заметку,
- назначить категорию,
- отобразить в расшифрованном виде,
- скопировать,
- удалить.

18.3 Локальные файлы

Файлы управляются локально через механизмы Browser/Client Storage, например OPFS, и хранятся в зашифрованном виде.

18.4 Cloud Synchronization

Для поддерживаемых файлов клиент может загружать на узел chunks, зашифрованные на стороне клиента. Сервер видит зашифрованные chunks и метаданные, но не расшифрованное содержимое файла.

Текущий UI применяет дополнительные ограничения Size/TTL к некоторым функциям синхронизации. Перед релизом эти пределы следует синхронизировать с конфигурацией сервера и README.

18.5 Cross-Device Sync

Полная церемония распределения ключей и Recovery для бесшовной Cross-Device синхронизации GaiaDrive остается известным Beta-ограничением.

19. GaiaDrop

GaiaDrop предназначен для внешних текстовых обращений на GaiaID.

19.1 Отправка без обычного входа в аккаунт

Из Authentication/Public Entry Point:

1. Выберите GaiaDrop.
2. Введите GaiaID получателя.
3. Напишите текст.
4. Отправьте обращение.

Текущий Release Path **ориентирован на текст**. GaiaDrop с вложениями пока не следует считать выпущенной функцией.

19.2 GaiaDrop Inbox

Получатель открывает область GaiaDrop и загружает свои обращения.

Детали могут включать:

- Timestamp
- Drop ID
- Payload Hash
- Sender Label или анонимизированный статус, если доступен
- текст после client-side decryption

19.3 Граница безопасности понятия "anonymous"

GaiaDrop предназначен для низкопороговых обращений, не обязательно привязанных к обычному аккаунту. Это нельзя рекламировать как математическую гарантию сетевой анонимности. Network- и Operational Metadata находятся вне такой гарантии.

20. Network Health

Network Health показывает агрегированное состояние сети/узлов.

Типичные данные:

- Accounts
- Identities
- Nodes
- Rooms
- Messages за временной интервал
- GaiaDrop Activity
- Federation Activity
- Uptime
- Protocol Version
- Crypto/Capability Status
- Federation Status
- SMTP Bridge Status

В Profile предусмотрена Privacy-настройка для анонимной или агрегированной Telemetry. Персональные Communication Graphs или GaiaIDs не должны считаться обычной публичной Health Metric.

21. Security Center / GaiaShield

Security Center - интерфейс пользователя и оператора для GaiaShield и связанных функций безопасности.

21.1 Protection Status

В зависимости от прав здесь могут отображаться:

- Account Protection
- открытые предупреждения
- неудачные попытки входа
- Message/Replay Anomalies
- Security Checks

После проверки предупреждения можно подтвердить.

21.2 Security Log

Security Events можно фильтровать по Severity/Category и экспортировать для анализа, например в JSON или CSV.

21.3 Gaia Passport

Область Passport отображает, среди прочего, Human Proof и Trust Information. Human Proof можно обновить.

21.4 Node Security

Операторы получают дополнительные события и статистику, относящиеся к узлу.

По дизайну GaiaShield фиксирует в том числе:

- Authentication Attacks
- Malformed Requests
- BOLA/BFLA Attempts
- Storage ACL Violations
- Federation Anomalies
- SMTP Abuse
- Governance Abuse
- GSN Spam
- Upload Abuse

Чувствительные поля редактируются или хешируются перед персистентностью/отображением там, где это предусмотрено Security Subsystem.

21.5 Nodesystem

Область Nodesystem показывает Registry/Node Information и диагностические действия.

Production-правило: не provision'ить Node Secrets преимущественно через Browser UI. Для production-узла использовать `gaiacom setup` и `gaiacom doctor`.

22. Abuse Center, Governance и Gaia's Eyes

22.1 Роли

В зависимости от Bootstrap/Credential Model Governance Layer различает роли вроде:

- User
- Reviewer
- Senior Reviewer
- Node Operator

Права ролей должны проверяться на стороне сервера.

22.2 Собственные Reports

Пользователь видит отправленные Cases и их статус. В зависимости от Workflow можно подать Appeal.

22.3 Reviewer Queue

Reviewer'ы получают Policy-based Cases и после server-side проверки прав могут подписывать оценки или предлагаемые действия.

Возможные меры:

- reject
- quarantine
- suspend

Точная Threshold Logic зависит от Policy/Governance-конфигурации.

22.4 Gaia's Eyes

Gaia's Eyes - не "Godmode". Workflow предназначен для контролируемого, ограниченного Disclosure, когда затронутый Client или предусмотренная криптографическая Disclosure Logic предоставляет необходимый материал.

22.5 Действия Node Operator

Операторы могут применять Node/Abuse Actions в своей Governance Scope и публиковать Transparency Information.

23. Профиль и персональные настройки

23.1 Редактирование профиля

Возможные поля:

- Display Name
- опциональное Real Name
- Website
- Bio/Status
- Avatar

Uploads и содержимое Profile должны оставаться sanitized или validated как на Client, так и на Server Side.

23.2 Смена пароля

Требуются:

- текущий пароль,
- новый пароль,
- подтверждение.

Server Password - не то же самое, что Recovery Mnemonic или локальный GaiaDrive/Export Password.

23.3 Устройства

См. главу 9. Sessions и Crypto Devices управляются из Profile.

23.4 Ключи

После Reauthentication, в зависимости от UI, клиент может показывать локальные Private Keys и Mnemonic.

Эта область особенно чувствительна. Screen Sharing, Browser Extensions, Malware или физический доступ к разблокированному устройству могут обойти предусмотренную защиту.

23.5 Экспорт Recovery Data

Recovery-файлы экспортируются в зашифрованном виде. Используйте отдельный сильный пароль и храните файл Offline либо отдельно от пароля.

23.6 Crypto Lock

Настраиваемые элементы могут включать:

- продолжительность Crypto Session,
- Inactivity Auto-Lock,

- PIN Unlock,
- WebAuthn PRF.

23.7 Privacy

Пользователь может настроить участие в анонимизированной/агрегированной Network Statistics.

23.8 Accessibility

Клиент предоставляет, среди прочего:

- Font Scale: 90%, 100%, 112%, 125%
- High Contrast
- Reduce Motion

23.9 Notifications

В зависимости от клиента:

- Notifications On/Off
- Content Preview
- Quiet Hours
- Start/End Time

23.10 Язык

Глобальный i18n-слой предоставленного Frontend содержит 18 языковых опций:

Deutsch, English, Русский, Español, Français, فارسی, 日本語, Português, العربية, 简体中文, हिन्दी, Türkçe, Italiano, Polski, Українська, 한국어, Indonesia и Shqip.

Некоторые поддиалоги пока предлагают меньшие Locale Lists, чем глобальный клиент.

23.11 Theme

Light/Dark Mode переключается через Navigation.

23.12 Удаление аккаунта

Danger Area требует явного подтверждения и текущего пароля.

Удаление следует считать необратимым. Необходимые Recovery/Export Data нужно сохранить заранее.

24. Горячие клавиши

Клиент содержит Gmail-style Navigation/Work Shortcuts. Они не срабатывают, когда пользователь вводит текст в соответствующих полях ввода.

Комбинация	Действие
g, затем i	Inbox
g, затем s	Sent
g, затем c	Chat
g, затем g	Groups
g, затем p	Profile
c	Compose
r	Reply
f	Forward
e	Archive
Delete, Backspace или #	Trash
s	Toggle Star

Часть II - Руководство оператора узла и развертывания

25. Модель эксплуатации

В рекомендуемом Linux-варианте production-узел GaiaCom состоит из:



Порт 8080 в стандартной production-модели **не должен быть напрямую открыт в Internet**.

26. Предварительные требования

26.1 Build System

В зависимости от собираемого компонента:

- Go Toolchain, соответствующий репозиторию (`go 1.25.0` , примечание toolchain `go1.26.5`)
- Node.js согласно `.node-version`
- npm
- Rust/Cargo
- `wasm-pack` для WASM
- Tauri Toolchain для Desktop
- Python для Hygiene/Security Scripts
- Android Studio/Gradle/JDK для Android

26.2 Production-сервер

Для предоставленного Deployment Path рекомендуется:

- Linux AMD64
 - NGINX
 - systemd
 - TLS Certificate
 - выделенный системный пользователь `gaiacom`
 - локальный SQLite на persistent storage
 - отдельный persistent GaiaCom Storage Path
 - DNS/FQDN
-

27. Локальная среда разработки

27.1 Репозиторий

Распакуйте или клонируйте репозиторий и работайте из его корня.

27.2 Окружение Backend

Backend не загружает `.env` автоматически, как типичное приложение с `godotenv`. Переменные должны быть экспортированы в окружение процесса либо загружены Shell/Service.

Минимальный **локальный** пример:

```
cd Backend

export GAIACOM_DEV_MODE=true
export SERVER_BIND_ADDRESS=127.0.0.1
export SERVER_PORT=8080
export DB_DRIVER=sqlite
export DB_PATH="$PWD/gaiacom-dev.db"
export GAIACOM_COOKIE_SECURE=false

go mod verify
go test ./...
go run ./cmd/gaiacom
```

В Dev Mode production-secrets нельзя повторно использовать как реальные Deployment Secrets.

27.3 Frontend

Для имеющейся Dev-Origin-конфигурации Backend более надежным путем является порт `3000`:

```
cd Frontend/frontend
npm ci --include=optional --ignore-scripts --no-audit --no-fund

VITE_API_URL=http://localhost:8080 \
npx vite --host localhost --port 3000
```

Эквивалент для PowerShell:

```
cd Frontend\frontend
npm ci --include=optional --ignore-scripts --no-audit --no-fund
$env:VITE_API_URL='http://localhost:8080'
npx vite --host localhost --port 3000
```

27.4 Почему не просто `npm start`?

Имеющийся скрипт:

```
vite --host 127.0.0.1
```

Без явного указания порта Vite может использовать стандартный порт, в то время как Dev-CORS Backend разрешает другие Origins. Поэтому воспроизводимая инструкция разработчика должна явно задавать Host и Port.

28. Production-сборка

28.1 Frontend

Bash:

```
cd Frontend/frontend
npm ci --include=optional --ignore-scripts --no-audit --no-fund
npm test
npm run test:hqc-browser
VITE_API_URL=https://node.example.org npm run build
```

PowerShell:

```
cd Frontend\frontend
npm ci --include=optional --ignore-scripts --no-audit --no-fund
$env:VITE_API_URL='https://node.example.org'
npm test
npm run build
npm run test:hqc-browser
```

Результат:

```
Frontend/frontend/dist/
```

28.2 Backend Linux AMD64

```
cd Backend
go mod verify
go test ./...

VERSION=2.0.0
COMMIT="$(git rev-parse HEAD)"
BUILT_AT="$(date -u +%Y-%m-%dT%H:%M:%SZ)"

CGO_ENABLED=0 GOOS=linux GOARCH=amd64 \
go build -trimpath -buildvcs=false \
-ldflags="-s -w \
-X gaiacom/backend/buildinfo.Version=${VERSION} \
-X gaiacom/backend/buildinfo.Commit=${COMMIT} \
-X gaiacom/backend/buildinfo.BuiltAt=${BUILT_AT}" \
-o gaiacom-backend-linux-amd64 ./cmd/gaiacom
```

Release Metadata имеет значение для безопасности и эксплуатации production-пути. Production-сервер не следует запускать с некорректным Development Release Marker.

29. Подготовка Linux-сервера

Пример для Debian/Ubuntu:

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y nginx certbot python3-certbot-nginx

sudo useradd -r -s /bin/false -d /opt/gaiacom gaiacom

sudo install -d -o root -g root -m 0755 /opt/gaiacom
sudo install -d -o root -g root -m 0755 /var/www/gaiacom
sudo install -d -o root -g root -m 0700 /etc/gaiacom
sudo install -d -o gaiacom -g gaiacom -m 0700 /var/lib/gaiacom
sudo install -d -o gaiacom -g gaiacom -m 0700 /opt/gaiacom/storage
```

Затем Backend Binary и Frontend следует передать контролируемым способом. Binary и Frontend не должны быть доступны для записи непривилегированному Runtime User, если это не строго необходимо.

30. Production-конфигурация с `gaiacom setup`

30.1 Канонический Setup

```
sudo /opt/gaiacom/gaiacom-backend setup \
--config /etc/gaiacom/gaiacom.env \
--server-name node.example.org \
--db-path /var/lib/gaiacom/gaiacom.db \
--port 8080
```

После этого:

```
sudo chmod 600 /etc/gaiacom/gaiacom.env
sudo chown root:root /etc/gaiacom/gaiacom.env
```

30.2 Что задает или создает `setup`

Текущий Provisioning Path устанавливает, среди прочего:

```
GAIACOM_DEV_MODE=false
GAIACOM_SERVER_NAME=<FQDN>
SERVER_BIND_ADDRESS=127.0.0.1
GAIACOM_ALLOW_PUBLIC_BIND=false
DB_DRIVER=sqlite
GAIACOM_COOKIE_SECURE=true
```

и создает либо сохраняет валидные Secrets для:

- JWT
- GaiaShield
- Metrics
- TrustMesh Epoch
- Ed25519 Server Identity

Configuration File создается атомарно с ограничительными правами.

30.3 Идемпотентность

`setup` спроектирован так, чтобы валидные существующие Secrets не заменялись при каждом запуске. Несовпадение Server Name или невалидные существующие Secrets не должны молча перезаписываться.

Поэтому: не генерируйте новые Secrets перед каждым стартом "на всякий случай".

31. gaiacom doctor и gaiacom version

31.1 Doctor

```
sudo /opt/gaiacom/gaiacom-backend doctor \
  --config /etc/gaiacom/gaiacom.env
```

Doctor проверяет, среди прочего:

- синтаксис конфигурации,
- силу/формат Secrets,
- Server Identity,
- каталог Database,
- File Permissions,
- критические Production Settings.

В Unix production-файл окружения не должен иметь лишних Group/World Permissions.

31.2 Version

```
/opt/gaiacom/gaiacom-backend version
```

Вывод включает Release/Protocol/Build Information: Version, Commit и Build Time.

32. Bootstrap Governance

Для начального Node Operator предусмотрена Governance Configuration.

Пример:

```
{  
  "bootstrap_gaia_id": "@operator:node.example.org"  
}
```

Сохранить как:

```
/etc/gaiacom/governance.json
```

Установить ограничительные права:

```
sudo chown root:root /etc/gaiacom/governance.json  
sudo chmod 600 /etc/gaiacom/governance.json
```

Добавить путь конфигурации в Environment:

```
echo 'GAIACOM_GOVERNANCE_CONFIG=/etc/gaiacom/governance.json' \  
| sudo tee -a /etc/gaiacom/gaiacom.env >/dev/null
```

После запуска Bootstrap GaiaID должен фактически существовать как подходящая идентичность либо пройти предусмотренный Bootstrap Flow.

33. Усиление systemd

Репозиторий содержит усиленный `deploy/systemd/gaiacom.service`.

Предусмотренные меры защиты включают:

- непривилегированный пользователь `gaiacom`
- `NoNewPrivileges=true`
- `PrivateTmp=true`
- `PrivateDevices=true`
- `ProtectSystem=strict`
- `ProtectHome=true`
- ограничительные Capability Boundaries
- `LockPersonality=true`
- `RestrictSUIDSGID=true`
- `UMask=0077`

После установки:

```
sudo systemctl daemon-reload
sudo systemctl enable --now gaiacom
sudo systemctl status gaiacom
```

Logs:

```
sudo journalctl -u gaiacom -f
```

34. NGINX, TLS и Reverse Proxy

34.1 Routing

NGINX должен:

- отдавать `/` из собранного Frontend,
- проксировать `/api/` на `127.0.0.1:8080`,
- внутренне передавать `/livez` и `/readyz`,
- передавать `/.well-known/gaiacom/` в Backend,
- не публиковать `/metrics` наружу.

34.2 Security Headers

Предоставленный VHost задает строгие Headers, в том числе:

- Content-Security-Policy
- X-Frame-Options: DENY
- X-Content-Type-Options: nosniff

CSP нельзя бездумно ослаблять широкими `*`, `unsafe-inline` или дополнительными Third-Party Origins, поскольку Web Client является частью криптографического Trust Path.

34.3 TLS

Настройте TLS Certificate через Certbot или эквивалентную инфраструктуру и контролируйте автоматическое продление.

35. Firewall и сетевая экспозиция

Минимально публично доступны:

- SSH, если требуется,
- TCP 80 для Redirect/ACME,
- TCP 443 для GaiaCom HTTPS.

Пример UFW:

```
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
```

Не открывать публично: Backend Port `8080`, если только намеренно не применяется иная, отдельно усиленная модель эксплуатации.

36. Node Registry и контролируемая федерация

36.1 Предварительные требования

Узлу необходимы:

- валидный FQDN,
- DNS,
- HTTPS,
- Server Signing Key,
- корректные Well-Known Endpoints,
- Registry/Authority Configuration,
- валидная Release/Core Information.

36.2 Well-Known Endpoints

Ключевые Federation Discovery/Transport Paths:

```
GET /.well-known/gaiacom/server
GET /.well-known/gaiacom/nodeinfo
GET /.well-known/gaiacom/nodes
POST /.well-known/gaiacom/s2s/v1/forward
```

36.3 Присоединение

Типовой процесс:

1. Развернуть узел.
2. Успешно выполнить `setup` и `doctor`.
3. Включить Service и TLS.
4. Указать Bootstrap Operator GaiaID.
5. Из Nodesystem выполнить Ping Main Node / Registry.
6. Проверить Admission/Quarantine/Block Status.
7. Отправить федеративное тестовое сообщение.

36.4 Controlled Federation

Sovereign Beta v2 **не является свободно открытой федерацией**. Узлы допускаются контролируемо.

36.5 S2S Security

Federation Path содержит защиту от:

- невалидных подписей,

- Replay,
 - неверных временных окон,
 - манипуляции Body Hash,
 - неверного Target,
 - SSRF к private/disallowed сетям.
-

37. Хранилище: локальный Object Store и S3/MinIO

37.1 Локальный Object Store

По умолчанию Backend Storage может хранить зашифрованные Chunks в локальной файловой системе.

Конфигурационно значимые переменные:

```
GAIACOM_STORAGE_ROOT  
GAIACOM_STORAGE_USER_QUOTA_BYTES  
GAIACOM_STORAGE_PENDING_TTL_HOURS
```

Код Object Store содержит Path-Jail/Path-Validation Logic. Пользовательские адаптеры не должны обходить этот слой защиты.

37.2 S3/MinIO

Опционально:

```
GAIACOM_OBJECT_STORE=s3  
GAIACOM_S3_ENDPOINT  
GAIACOM_S3_BUCKET  
GAIACOM_S3_REGION  
GAIACOM_S3_ACCESS_KEY  
GAIACOM_S3_SECRET_KEY  
GAIACOM_S3_PREFIX  
GAIACOM_S3_PATH_STYLE
```

Даже при S3 Object Store должен получать только Chunk Data, уже зашифрованные на стороне клиента.

37.3 Модели доступа

Storage Metadata поддерживает Owner/Grant/Public-style ACL States, а также Expiration/Revocation Paths.

"Public" не означает, что объект, зашифрованный на стороне клиента, автоматически становится публично доступным в расшифрованном виде; модели доступа и криптографии нужно рассматривать совместно.

38. Production-настройка SMTP

Релевантные переменные в текущем состоянии репозитория/документации:

```
GAIACOM_SMTP_HOST  
GAIACOM_SMTP_PORT  
GAIACOM_SMTP_FROM  
GAIACOM_SMTP_USERNAME  
GAIACOM_SMTP_PASSWORD  
GAIACOM_SMTP_INGEST_TOKEN  
GAIACOM_SMTP_TLS_MODE
```

Типичный порт - 587 для STARTTLS, в зависимости от Relay.

Production-проверка:

1. Relay аутентифицирован?
 2. TLS обязателен?
 3. From Domain корректен?
 4. SPF настроен?
 5. DKIM подписывает?
 6. DMARC активен?
 7. Bounce/Abuse Behavior контролируется?
 8. Ingest Token сильный и секретный?
 9. SMTP Endpoint имеет Rate Limit?
 10. Downgrade видим в UI?
-

39. Health Checks, Metrics и Logs

39.1 Liveness

```
curl --fail --silent https://node.example.org/livez
```

39.2 Readiness

```
curl --fail --silent https://node.example.org/readyz
```

39.3 Nodeinfo

```
curl --fail --silent \  
https://node.example.org/.well-known/gaiacom/nodeinfo
```

39.4 Nodes

```
curl -s https://node.example.org/.well-known/gaiacom/nodes
```

39.5 Public Version

`/api/v1/public/version` должен отражать фактически собранные Version, Commit ID и Build Time.

39.6 Metrics

В предусмотренной конфигурации `/metrics` **не является публичным**. Локально либо через авторизованный Monitoring Access Endpoint требует Metrics Token. При отсутствии или неверном Token защитный путь не должен раскрывать лишние детали.

39.7 Journald

```
sudo journalctl -u gaiacom --since today  
sudo journalctl -u gaiacom -f
```

Secrets или открытый Key Material не должны попадать в Logs.

40. Резервное копирование и восстановление

40.1 SQLite

Перед обновлением создайте согласованную резервную копию SQLite, например через SQLite Backup Function или `.backup`.

Пример:

```
sudo systemctl stop gaiacom
sqlite3 /var/lib/gaiacom/gaiacom.db \
    ".backup '/var/backups/gaiacom/gaiacom-$(date +%F-%H%M%S).db'"
sudo systemctl start gaiacom
```

40.2 Migration Snapshots

Внутренний Migration Manager создает Backups перед ожидающими Schema Migrations. Они **не заменяют полноценную стратегию резервного копирования**.

40.3 Object Store

Database и Object Store необходимо резервировать согласованно. Копия Database без соответствующих Chunk Objects может быть неполной.

40.4 Secrets

Production Secrets следует резервировать отдельно и в зашифрованном виде:

- Environment
- Governance Configuration
- необходимые server-side Identity Keys

Потеря Server Identity или Trust-related Secrets может иметь последствия для Federation/Verification.

40.5 Client Recovery

Server Backup не заменяет User Recovery. Private Client Keys/Mnemonics намеренно не хранятся на сервере в виде plaintext Godmode Backup.

41. Обновления и Rollback

41.1 До обновления

- Проверить Release Notes.
- Выполнить Tests/Gates в сборке.
- Создать Backup Database.
- Создать Backup Object Store.
- Создать Backup Secrets/Configuration.
- Архивировать текущий Binary/Frontend Build.

41.2 Обновление

1. Остановить Service.
2. Установить новый Backend Binary.
3. Атомарно или контролируемо развернуть новый Frontend.
4. Запустить Service.
5. Проверить `livez`, `readyz`, `nodeinfo`, Version и Federation.
6. Проверить Logs на Migration/Runtime Errors.

41.3 Rollback

Binary Rollback безопасен только тогда, когда более старый Binary совместим с уже мигрированной Database Schema.

При несовместимой Migration необходимо восстановить **предыдущий Database Backup вместе с соответствующим состоянием Object Store**.

42. Production-чек-лист

Перед публичной эксплуатацией:

- ☐ FQDN и DNS корректны.
 - ☐ TLS валиден, Auto-Renewal проверен.
 - ☐ Backend привязан только к Loopback.
 - ☐ `GAIACOM_ALLOW_PUBLIC_BIND=false`, если намеренно не используется иная модель.
 - ☐ `gaiacom setup` успешен.
 - ☐ `gaiacom doctor` не показывает критических ошибок.
 - ☐ Configuration File имеет mode `0600`.
 - ☐ Secrets сильные и не хранятся в Git.
 - ☐ Governance Bootstrap корректен.
 - ☐ systemd Hardening активен.
 - ☐ NGINX Hardening активен.
 - ☐ `/metrics` недоступен извне.
 - ☐ `livez` / `readyz` успешны.
 - ☐ Version/Commit/Build Time корректны.
 - ☐ SQLite/Object Store Backup протестирован.
 - ☐ SMTP TLS/SPF/DKIM/DMARC проверены, если SMTP включен.
 - ☐ Federation протестирована контролируемо.
 - ☐ Browser CSP не ослаблен.
 - ☐ Security/Hygiene Gates успешны.
 - ☐ Лицензирование AGPLv3 согласовано в `LICENSE`, README, Package Metadata и Source Notices.
 - ☐ Невыпущенные Beta-функции не рекламируются как Production Ready.
-

Часть III - Руководство разработчика

43. Структура репозитория

Упрощенно:

```
GaiaCom-main/
├── Frontend/frontend/      React 18 / Vite Client
├── Backend/                Go Backend
├── SovereignCryptoCore/    Rust Crypto Core
├── SovereignCryptoWasm/    WASM Bindings
├── DesktopClient/          Tauri 2 Desktop Shell
├── Core/rust/gaiacore/     Zero-/Low-Dependency Core Primitives
├── Android/                Kotlin/Compose модульная Mobile Platform
├── deploy/                 NGINX + systemd
├── security/               PoCs / adversarial tests
├── scripts/                Hygiene / Architecture Gates
├── docs/                   Operations, Security, Federation, Runbooks
├── README.md
├── SECURITY.md
├── SECURITY_INVARIANTS.md
└── ARCHITECTURE.md         краткий файл архитектуры репозитория
```

Отдельно предоставленная большая карта архитектуры значительно подробнее и полностью включена в Приложение к этому руководству.

44. Архитектура Frontend

44.1 Stack

- React 18
- Vite
- Lazy Loading / Suspense
- Web Worker
- WebAssembly
- OPFS
- PWA Service Worker
- локальный i18n Layer

44.2 Центральные Entrypoints

```
Frontend/frontend/src/index.jsx
Frontend/frontend/src/App.jsx
Frontend/frontend/src/api.js
Frontend/frontend/src/crypto.js
Frontend/frontend/src/sovereignCrypto.js
```

44.3 UI Orchestration

`App.jsx` управляет глобальным UI/Session State. `AppMainContent` загружает активную Pane. `AppModalLayer` централизует поверхности Modal/Dialog.

44.4 Важные Panes

- DashboardPane
- ChatPane
- GroupChatPane
- PublicChannelsPane
- ReaderPane
- ComposerPane
- ProfilePane
- DrivePane
- DropPane
- GsnPane
- SecurityCenter
- AbuseCenter
- NetworkHealthDashboard

44.5 Hooks

Domain Logic инкапсулирована, среди прочего, в:

- `useGaiaAuth`
- `useChat`
- `useEmails`
- `useDrive`
- `useGaiaDrop`
- `usePublicChannels`
- `useGsn`
- `usePresence`
- `useKeyChangeDetection`
- `useInactivityLock`

44.6 Crypto Provider

`sovereignCrypto.js` абстрагирует между:

- Web Provider с изолированным Crypto Worker/WASM
- Native Provider через Tauri IPC

Provider Path не должен незаметно деградировать, если требуемые Cryptographic Primitives недоступны.

45. Архитектура Backend

45.1 Stack

- Go
- `net/http` через собственный `Backend/httpx`
- pure-Go SQLite
- domain-oriented packages
- без Gin/GORM в текущем Architecture Path

45.2 Entrypoint

```
Backend/cmd/gaiacom/main.go
```

делегирует выполнение в Backend Lifecycle.

45.3 Основные домены

```
auth/  
devices/  
identity/  
messaging/  
mailbox/  
room/  
publicchannels/  
gsn/  
storage/  
gaiaDROP/  
smtpbridge/  
federation/  
noderegistry/  
trustmesh/  
governance/  
internal/security/  
networkhealth/  
operations/  
provision/  
mobileapi/
```

45.4 Security Middleware

`httpx`, GaiaShield и domain-specific Authorization Checks образуют несколько слоев. Защита BOLA/BFLA не должна существовать только во Frontend.

45.5 Scheduler

`scheduler.go` обрабатывает запланированные Draft/Send Actions.

46. Персистентность и SQLite

46.1 Режим Database

Database Layer использует pure-Go SQLite и настраивает, среди прочего:

- WAL
- Foreign Keys
- Busy Timeout
- `synchronous=FULL`

46.2 Repository Layer

`Backend/repository` предоставляет Store Interfaces и SQL Implementations.

Domain Adapters покрывают, среди прочего:

- Auth
- Devices
- Identity
- Messaging
- Mailbox
- Rooms
- Public Channels
- GSN
- Storage/GaiaDrop
- Federation
- Governance
- Security Events
- Rate Limits
- Presence
- Offline Transport Queue

46.3 Migrations

Migrations используют Ledger/Checksum Model и создают Database Snapshots перед ожидающими Migrations.

46.4 Граница масштабирования

Beta-эксплуатация рассчитана на **один Backend Process на одну SQLite Database**. SQLite в данном дизайне не является активным Multi-Primary/Horizontal-Scale Database Cluster.

47. Sovereign Crypto Core, WASM и Desktop

47.1 Hybrid KEM

Дизайн объединяет:

- ephemeral X25519
- ML-KEM-1024
- HQC-256

через общую Key Derivation с SHA3/HKDF Binding.

47.2 Accelerated Cascade

Дизайн:

- Twofish-256-EAX
- AES-256-GCM
- Ed25519 Signature

47.3 Top Secret Cascade

Дизайн:

- Serpent-256-CTR + HMAC-SHA3-512
- Twofish-256-EAX
- XChaCha20-Poly1305
- AES-256-GCM-SIV
- Ed25519
- ML-DSA-87

47.4 SovereignCryptoCore

Rust-реализация Cipher Cascades и контролируемого Memory Zeroization.

47.5 WASM

`SovereignCryptoWasm` экспортирует Rust Primitives для Web Client. Browser HQC дополнительно работает через локальный изолированный Provider/Worker Path.

47.6 Desktop Tauri

Tauri Host намеренно предоставляет узкую IPC Surface:

```
sovereign_hqc256_keypair
sovereign_hqc256_encapsulate
```

```
sovereign_hqc256_decapsulate  
sovereign_seal  
sovereign_open
```

Узкий Bridge уменьшает количество native-функций, которые должны быть доступны из WebView.

47.7 Desktop Custom Node

В текущем снимке Desktop API Origin закреплен на <https://beta.gaiacom.de>. Для собственного узла необходимо проверить и изменить как Origin Logic, так и Tauri CSP, после чего пересобрать клиент.

48. Android-платформа

Android-структура модульная:

```
:app
:platform:contracts
:platform:kernel
:platform:node
:platform:security
:platform:security-android
:platform:security-auth
:platform:runtime
:platform:remote
:platform:api
:platform:identity
```

Запланированные/реализованные основы включают:

- Jetpack Compose UI
- Embedded Go Node
- Android Keystore
- StrongBox
- Biometrics/PIN
- Typed API Facade
- основы Wi-Fi Direct/BLE Transport Adapters

Статус: Technical Beta. Согласно границам релиза, полный Mobile Sovereign v1 Message Path пока нельзя считать полностью Production Ready.

49. Обзор API

Полная справка по Endpoints находится в карте архитектуры в Приложении А. Для повседневной эксплуатации достаточно следующего Domain Overview.

49.1 Auth

```
/api/v1/auth/register  
/api/v1/auth/login  
/api/v1/auth/refresh  
/api/v1/auth/logout  
/api/v1/auth/status  
/api/v1/auth/change-password  
/api/v1/auth/delete-account
```

49.2 Devices

Device Sessions, Device Keys и Pairings под `/api/v1/devices/...`.

49.3 Identity

Создание идентичности, собственная идентичность, публичная идентичность, Human Proof и Trust Passport.

49.4 Messaging

Send, Inbox, Read, Edit, Delete, Reactions, Proofs и Presence.

49.5 Mailbox

Message State, Drafts, Labels, Filters, Mailbox Settings и Global Search.

49.6 Rooms

Rooms, Members, Channels, Join Requests, Invitations и Moderation.

49.7 Public Channels

Channels, Posts, Comments, Reactions, Subscriptions и Moderation.

49.8 GSN

Feed, Posts, Comments, Reactions, Profiles и Follows.

49.9 Storage

Upload Init, Chunks, Complete, Download, Grants и Metadata.

49.10 GaiaDrop

Public Submit Path плюс аутентифицированные Inbox/Read/Delete-функции.

49.11 SMTP

Outbound Send и защищенный Ingest.

49.12 Governance / Reports

Reports, Reviewer, Node Abuse, Disclosure и Gaia's Eyes.

49.13 Security / Node

Security Events, Node Security, Registry и Operator Actions.

49.14 Public / Operations

Network Health, Version, Transparency, `livez`, `readyz`, защищенные Metrics.

49.15 Federation

Well-Known Discovery и подписанный S2S Forwarding.

50. Security- и Quality-Gates

Release Path в README документирует Gate Run для Backend, Frontend, Rust, WASM, Tauri и Repository Hygiene.

Пример:

```
cd Backend
go mod verify
go test ./...

cd ../Frontend/frontend
npm ci --include=optional --ignore-scripts --no-audit --no-fund
npm test
npm run build
npm run test:hqc-browser

cd ../../SovereignCryptoCore
cargo test --locked

cd ../SovereignCryptoWasm
cargo test --locked
wasm-pack build --target web --release

cd ../DesktopClient/src-tauri
cargo test --locked

cd ../../
python scripts/repository_hygiene.py
```

Дополнительно существуют наборы:

```
security/master_security_suite.py
security/total_system_poc_runner.py
security/poc/
security/extreme/
scripts/android_architecture_gate.py
```

PoC/Adversarial Areas покрывают, среди прочего:

- Federation SSRF
- Replay
- Signature Verification
- GaiaDrop Size/XSS/Rate-Limit Behavior
- GaiaProof
- Room Authorization
- Governance
- Key History
- Secure Disclosure
- SMTP Injection/Downgrade
- Trust Passport
- BOLA/BFLA

- Database/Concurrency Stress

Примечание: эти команды являются Release Gate, задокументированным в репозитории. Руководство не утверждает, что при подготовке документации абсолютно каждая Toolchain-проверка была полностью запущена повторно.

51. Безопасная разработка и расширение

51.1 Сначала классифицируйте новую функцию

Перед реализацией спросите:

- Она касается открытого текста?
- Она касается Private Keys?
- Она касается Envelopes?
- Она касается Federation?
- Она касается Authorization?
- Она касается Disclosure/Governance?
- Она касается Object Storage?
- Она добавляет External Origin или Dependency?

Чем больше ответов "да", тем глубже должна быть Security Review.

51.2 Не реализуйте Security только в UI

Скрытая кнопка - не Access Control.

Authorization должна проверяться server-side для конкретного объекта и конкретного действия.

51.3 Никакого Silent Downgrade

Если отсутствует Top Secret, PQC Capability или ключ получателя:

- Сделайте ошибку видимой.
- Не переключайтесь прозрачно на более слабую Security.

51.4 Рассматривайте CSP как Security Boundary

Особенно в Web Client выдаваемое JavaScript-приложение является частью Trust Path. Новые CDN Dependencies, Inline Scripts или Third-Party Origins меняют Threat Model.

51.5 Минимизируйте Logs

Никогда не помещайте в Debug/Audit Logs:

- Private Keys
- Mnemonics
- Recovery Secrets
- JWT Secrets
- SMTP Passwords

- Plaintext Messages

51.6 Migrations

Schema Changes должны быть:

- migration-based,
- auditable,
- backwards-/rollback-aware,
- выполняться с Backups,
- тестироваться при Concurrent Access.

51.7 Новые Federation Paths

Дополнительно тестируйте:

- SSRF
 - Private IP Ranges
 - DNS Rebinding
 - Replay
 - Clock Skew
 - Body Hash
 - Signature
 - Target Confusion
 - Queue Poisoning
 - Resource Exhaustion
-

Часть IV - Эксплуатационная помощь

52. Troubleshooting

52.1 Frontend не может локально достучаться до Backend

Симптомы: CORS Errors, Login сразу завершается ошибкой, API сообщает "network error".

Проверить:

```
VITE_API_URL=http://localhost:8080
Frontend at http://localhost:3000
Backend at 127.0.0.1:8080
GAIACOM_DEV_MODE=true
```

Не смешивайте случайно `127.0.0.1:5173` и `localhost:3000`, если CORS Allowlist содержит лишь часть этих Origins.

52.2 `.env` существует, но, похоже, игнорируется

Backend не рассматривает файл автоматически как `.env` Loader Application.

Решение:

```
set -a
source /path/to/gaiacom.env
set +a
./gaiacom-backend
```

В Production systemd загружает `EnvironmentFile=` из предоставленного Unit.

52.3 Production-сервер не запускается из-за Build Info

Создайте Production Build с Version, Commit и Build Time. Не копируйте просто неквалифицированный Development Binary.

52.4 `doctor` жалуется на File Permissions

```
sudo chown root:root /etc/gaiacom/gaiacom.env
sudo chmod 600 /etc/gaiacom/gaiacom.env
```

52.5 `readyz` не проходит

Проверить:

- доступна ли Database,
- успешно ли завершились Migrations,
- доступен ли Storage Directory для записи,

- валидна ли Runtime Configuration,
- Logs: `journalctl -u gaiacom`.

52.6 `/metrics` возвращает 404

Это может быть намеренно. Metrics защищены; отсутствующий/неверный Bearer Token не должен публично раскрывать Endpoint. NGINX в любом случае должен блокировать его извне.

52.7 Federation работает локально, но не между узлами

Проверить:

- DNS
- TLS
- Server Name
- Registry Status
- Node Admission
- Well-Known Endpoints
- System Time
- Signing Key
- SSRF/Address Policy
- поддерживает ли Target Node требуемый Security Suite
- Quarantine/Block Status

52.8 Top Secret невозможно включить

При отсутствии обязательной Capability это корректно.

Проверить:

- Recipient Keyset
- HQC Provider
- ML-KEM Key
- Required Signature Keys
- Client Provider
- Key-Change Warning
- Node/Protocol Capability

Не "исправляйте" проблему удалением проверки.

52.9 Desktop Build всегда обращается к `beta.gaiacom.de`

Текущий снимок закрепляет Desktop Origin. Проверьте/измените `Frontend/frontend/src/api.js` и Tauri CSP, затем пересоберите.

52.10 Регистрация возможна несмотря на `OPEN_REGISTRATION=false`

В проверенном снимке эта переменная не связана с User Registration Service как однозначный Gatekeeper.

Если требуется закрытая регистрация пользователей, Auth Endpoint нужно явно закрыть server-side Gate и протестировать.

52.11 Chat показывает Attachment Functionality, хотя Release говорит "disabled"

UI содержит экспериментальные Paths. Для Sovereign Beta v2 определяющей является Release Matrix. Невыпущенные Attachment/Media Paths следует отключить либо явно пометить как Experimental.

52.12 Migration успешна, но Rollback Binary не запускается

Schema могла стать несовместимой. Восстановите соответствующий Database Backup вместе с соответствующим состоянием Object Store.

52.13 SMTP работает, но сообщения попадают в Spam

Проверить:

- PTR/rDNS
 - SPF
 - DKIM
 - DMARC
 - From Alignment
 - Relay Reputation
 - TLS
 - Rate
 - Content
 - Bounce Handling
-

53. Известные ограничения Sovereign Beta v2

1. **SQLite Deployment:** один Backend Process на Database; отсутствует активная Multi-Primary HA Database.
2. **Web-Origin Trust:** скомпрометированный узел теоретически может выдать Web Client измененный JavaScript до шифрования сообщения.
3. **Endpoint Compromise:** разблокированные устройства и скомпрометированные операционные системы остаются вне базового криптографического предположения.
4. **Нет гарантии анонимности:** шифрование содержимого не устраняет все Metadata.
5. **SMTP Downgrade:** традиционная электронная почта находится вне полной модели безопасности GaiaCom.
6. **Attachments:** Chat/Media Attachments пока не выпущены полностью.
7. **GaiaDrop:** Release Path является Text-Only; вложения еще не выпущены.
8. **Mobile:** Android-архитектура существует, но полный Sovereign v1 Mobile Message Path остается Technical Beta.
9. **Federation:** контролируемая, не открытая.
10. **Governance:** Cross-Node Policies нельзя интерпретировать как глобальный Consensus.
11. **GaiaDrive Cross Device:** Key-Share/Recovery Ceremony остается Beta-границей.
12. **Automatic Updates:** полностью выпущенной автоматизированной Update Pipeline пока нет.
13. **Recovery Operations:** Recovery-процессы необходимо организационно репетировать.
14. **Accessibility/Low-End Performance:** входят в Release Gates, но все еще являются Beta Quality Area.

Поэтому GaiaCom не следует рекламировать как "невзламываемый", "полностью анонимный", "High Availability без дополнительной архитектуры" или "невосприимчивый к компрометации Endpoints/Servers".

54. Глоссарий

AAD - Additional Authenticated Data; метаданные, которые могут быть криптографически связаны с Ciphertext.

BFLA - Broken Function Level Authorization.

BOLA - Broken Object Level Authorization.

CSP - Content Security Policy.

E2EE - End-to-End Encryption, сквозное шифрование.

Envelope - структурированный зашифрованный контейнер сообщения.

Federation - коммуникация между независимыми узлами GaiaCom.

GaiaID - федеративный адрес GaiaCom.

GaiaProof - формат криптографического доказательства.

GaiaShield - Security/Audit Subsystem.

Gaia's Eyes - контролируемый Disclosure/Reviewer Workflow.

GSN - GaiaSocialNetwork.

HQC-256 - ориентированный на постквантовую устойчивость KEM в Sovereign Design.

ML-DSA-87 - ориентированная на постквантовую устойчивость Signature Scheme.

ML-KEM-1024 - ориентированный на постквантовую устойчивость Key Encapsulation Mechanism.

OPFS - Origin Private File System в Browser.

PDU - Protocol Data Unit в Federation Transport.

PQC - Post-Quantum Cryptography.

S2S - Server-to-Server.

TrustMesh - локальная/policy-based система репутации.

Trust Passport - агрегированное представление Identity/Trust.

WASM - WebAssembly.

Zero Server Trust - принцип проектирования, при котором серверу не требуется расшифровывать конфиденциальный контент.

55. Сопровождение документации

При каждом релевантном изменении Release следует совместно проверять как минимум следующие документы:

```
README.md
SECURITY.md
SECURITY_INVARIANTS.md
DEPLOYMENT.md
docs/*
Frontend/frontend/package.json
Backend/go.mod
Backend/provision/*
deploy/nginx/*
deploy/systemd/*
```

Большую карту архитектуры также следует обновлять при изменении:

- Crypto Suites
- API Endpoints
- Tables
- Roles
- Modules
- Frontend Panes
- Federation Flows
- Deployment Requirements

Для будущих релизов рекомендуется четкая иерархия документации:

1. **Source Code + Tests** - фактическое поведение.
 2. **Release Contract / README** - выпущенные функции и границы.
 3. **Deployment Guide** - каноническая Production-эксплуатация.
 4. **Architecture** - углубленное техническое описание.
 5. **Manual** - объединение для пользователей и эксплуатации.
-

Приложение А - Полная карта архитектуры (перевод на русский язык)

Техническая карта системы и архитектуры GaiaCom

// STATUS: PLATIN VGT SUPREME

// REVISION: 2.0.0 Sovereign Beta v2

// АРХИТЕКТУРНАЯ ДОКУМЕНТАЦИЯ И СИСТЕМАТИЧЕСКОЕ ДЕРЕВО ДАННЫХ

Содержание (индекс архитектуры)

1. Обзор системы

- Философия архитектуры и Zero Server Trust
- Runtime-среды и технологический стек
- Глобальное дерево архитектуры

2. Привязка файлов к архитектуре

- Frontend Client (React / Vite)
- Desktop Shell (Tauri 2 Native Client)
- Sovereign Crypto Core & WebAssembly
- Core Rust Engine (Zero Dependency)
- Backend Application Layer (Go net/http & httpx)
- Персистентность и SQLite Database
- Android Embedded Node & Modular Platform
- Deployment, DevOps & Hardening
- Security PoCs, Audits & Quality Gates

3. Документация модулей

- Модуль 1: Sovereign Cryptographic Engine (Dual-PQC & Multi-Cipher Cascade)
- Модуль 2: Authentication, Sessions & Key Vault (GaiaVault / Auth)
- Модуль 3: Identity, GaiaID & Trust Passport
- Модуль 4: Quantum Chat & E2EE Direct Messaging
- Модуль 5: Group Chats, Rooms & Channels
- Модуль 6: GaiaMail & Mailbox State Management
- Модуль 7: SMTP Legacy Bridge & Downgrade Gateway
- Модуль 8: Public Channels
- Модуль 9: GSN - GaiaSocialNetwork
- Модуль 10: GaiaDrive & Encrypted Chunk Storage
- Модуль 11: GaiaDrop
- Модуль 12: Federation & S2S PDU Transport
- Модуль 13: Node Registry & Core-Hash Verification
- Модуль 14: TrustMesh & Local Reputation Scoring
- Модуль 15: Governance, Gaia's Eyes & Reviewer Workflows
- Модуль 16: GaiaShield Security Operations & Intrusion Defense
- Модуль 17: Desktop Client Integration (Tauri 2 Bridge)
- Модуль 18: Android Mobile Platform (Embedded Node)

4. Подробная документация Dashboard & Workspace

5. CSS & UI Architecture

6. API Architecture & Endpoint Reference

- 7. Системные потоки данных
 - 8. Shared & Core Components
 - 9. Архитектурные связи и Mermaid-диаграммы
 - 10. Ссылки на файлы
 - 11. Архитектурные особенности и неиспользуемые файлы
-

1. Обзор системы

1.1 Философия архитектуры и Zero Server Trust

- GaiaCom - это **федеративная коммуникационная инфраструктура с постквантовой защитой**, построенная на строгих принципах **Zero Server Trust** и **Client-Side Computing**:
- Zero Server Trust:** сервер (Go Backend) выступает исключительно как транспортный посредник, Routing Instance, Federation Node и слой хранения для зашифрованных Envelopes (`message_envelopes`), зашифрованных Chunks (`file_chunks`) и минимальных Identity/Audit Metadata. Ни в один момент сервер не имеет доступа к:
 - сообщениям или вложениям в открытом виде;
 - BIP-39 Mnemonics или Private Keys (X25519, ML-KEM-1024, HQC-256, Ed25519, ML-DSA-87);
 - симметричным ключам сообщений или Session Root Secrets;
 - расшифрованному содержимому локального хранилища (GaiaDrive / GaiaVault).
 - Dual-PQC Hybrid KEM:** шифрование основано на комбинаторе трех секретов (HKDF-SHA3-512), объединяющем классический Ephemeral X25519, постквантовый ML-KEM-1024 и постквантовый HQC-256.
 - Multi-Cipher Authenticated Cascade:**
 - *Accelerated Mode*: Twofish-256-EAX + AES-256-GCM (подписи Ed25519)
 - *Top Secret Mode*: четырехступенчатый каскад: Serpent-256-CTR-HMAC-SHA3-512 + Twofish-256-EAX + XChaCha20-Poly1305 + AES-256-GCM-SIV (подписи Ed25519 + ML-DSA-87)
 - Отсутствие административного Master Key (No Godmode):** ни операторы узлов, ни аудиторы не могут ретроспективно расшифровывать зашифрованные данные. Модерация основана на доказательствах с использованием криптографических Proofs (`GaiaProof`), Disclosure Tokens (`Gaia's Eyes`) и локальной федеративной репутации (`TrustMesh`).
 - Zero-Dependency Edge:** Gin и GORM полностью удалены из Backend. HTTP Routing выполняется через `Backend/httpx` (чистый Go `net/http`), а персистентность - через `pure-Go SQLite` (`modernc.org/sqlite`).

1.2 Runtime-среды и технологический стек

Слой	Технология	Назначение	Source Path
Web Frontend	React 18, Vite, Web Worker	Single-page application, UI, криптографический Browser Worker, OPFS	Frontend/frontend/
Desktop Shell	Tauri 2.11, Rust 1.85	Native Desktop App, изолированный IPC, native HQC и cascade-команды	DesktopClient/src-tauri/
Sovereign Crypto Core	Rust 1.85, zeroize	Общий Multi-Cipher Cascade, native & WASM compilation	SovereignCryptoCore/
Sovereign Crypto WASM	Rust, wasm-pack	WebAssembly Bridge для Browser Crypto Worker	SovereignCryptoWasm/
Core Primitives	Rust (Zero Dependency)	Детерминированная валидация (GaiaID, Envelope, TrustMesh)	Core/rust/gaiacore/

Слой	Технология	Назначение	Source Path
Backend Server	Go 1.26.5 (Linux AMD64)	Federation, Routing, SQLite Persistence, ACLs, GaiaShield, SMTP	Backend/
In-Memory Mobile Node	Go (Backend/mobileapi)	Embedded Node для Android/iOS через Gomobile Bridge	Backend/mobileapi/
Android Client	Kotlin, Jetpack Compose	Модульный Native Mobile Node с Keystore/StrongBox	Android/
Database	SQLite (pure-Go WAL Mode)	Локальная и server-side реляционная персистентность с Triggers	Backend/database/
Edge Proxy	NGINX	TLS Termination, CSP Headers, Reverse Proxy (:8080)	deploy/nginx/
Service Supervisor	systemd	Sandboxed Linux Service с минимальными привилегиями	deploy/systemd/

1.3 Глобальное дерево архитектуры

GAIACOM PLATFORM

├─ Dashboard & Workspace (Frontend UI) [Frontend/frontend/src/]

├─ Layout & Navigation

├─ NavigationSidebar.jsx (Основная навигация: Workspace, Network, Personal)

├─ ListPane.jsx (Списки: Mailbox, Rooms, Contacts, GaiaDrop, GaiaDrive)

├─ MobileNavigationDock.jsx (Нижний Mobile Dock для Touch Devices)

├─ LogoMark.jsx (SVG Branding и AstraeaOS Icon)

└─ VersionBadge.jsx (Индикатор Build и Consensus)

├─ Dashboard (Command Center)

└─ DashboardPane.jsx (System Metrics, Uplink Status, PWA Install, Quick Actions)

├─ Workspace Panes (Communication Cores)

├─ ChatPane.jsx (Quantum Chat: 1-to-1 E2EE PQC Messages)

├─ GroupChatPane.jsx (Group Chats: Rooms, Subchannels, Moderation)

├─ PublicChannelsPane.jsx (Public Channels: публичные Posts, Comments, Reactions)

├─ ReaderPane.jsx (Mail Reader: отображение Native и SMTP Mail)

└─ ComposerPane.jsx (Mail Composer: Draft Management, Scheduling, SMTP Warning)

├─ Network, Security & Governance Panes

├─ GsnPane.jsx (GaiaSocialNetwork: децентрализованный Feed, Profiles, Reactions)

├─ NetworkHealthDashboard.jsx (Network Health, Ping Latency, Federation Nodes)

├─ SecurityCenter.jsx (GaiaShield: Security Events, Audit Chain, Node Audit)

└─ AbuseCenter.jsx (Reporting Center: Case Reports, Gaia's Eyes, Reviewer Queue)

├─ Personal Areas & Storage

├─ ProfilePane.jsx (Profile Management, Mnemonics, Keys, Inactivity Lock)

├─ DrivePane.jsx (GaiaDrive: OPFS-зашифрованные Files, Local Notes, Cloud Sync)

├─ DropPane.jsx (GaiaDrop: анонимные Text Submissions & Inbox)

└─ VaultPane.jsx (Legacy GaiaVault Notes; заменен DrivePane)

├─ Modal & Dialog Layers

├─ AppModalLayer.jsx (Центральный Modal Manager)

├─ AppFeedbackModals.jsx (User Confirmations, Alerts, 3-way Dialogs)

├─ AuthScreen.jsx (Login, Registration, Account Recovery)

├─ UnlockScreen.jsx (Session Unlock через Password / PIN / WebAuthn)

├─ SetupWizard.jsx (First-Setup Wizard для новых идентичностей)

├─ FirstRunOnboarding.jsx (Вводный тур после первой установки)

├─ ContactProfileModal.jsx (Contact Detail Card, Trust Passport, Key History)

├─ AddContactModal.jsx (Ручной импорт контакта через GaiaID)

├─ CreateGroupModal.jsx (Создание Room с опцией Top Secret)

├─ JoinGroupModal.jsx (Вход в Room через Hash или Invitation Link)

├─ CreateChannelModal.jsx (Создание Subchannel в существующей Room)

├─ GroupSettingsModal.jsx (Room Settings, Slow Mode, Member Management)

├─ KeyChangeWarningModal.jsx (Security Warning при изменении ключа получателя)

├─ QuantumShieldModal.jsx (Объяснение и Status Display Quantum Protection)

└─ HumanProofDialog.jsx (Криптографический Proof-of-Human Evidence)

- └─ Styling Architecture [Frontend/frontend/src/styles/]
 - └─ 27 каскадных CSS Files (base, theme, layout, chat, nebula, ui, etc.)
 - └─ startup.css (Критические Pre-Hydration Styles для исключения FOUC)
- └─ Sovereign Cryptographic Engine (Dual-PQC & Multi-Cipher)
 - └─ SovereignCryptoCore/ (Rust Cascade: Twofish, AES-GCM, Serpent, XChaCha20, AES-GCM-SIV)
 - └─ SovereignCryptoWasm/ (WASM Export для Web Browsers)
 - └─ DesktopClient/src-tauri/ (Tauri 2 Native Host с pqcrypto-hqc & IPC Bridge)
 - └─ Core/rust/gaiacore/ (Zero-Dependency Rust Primitives: GaiaID, Envelopes, TrustMesh)
 - └─ Frontend/frontend/src/sovereign/
 - └─ nativeProvider.js (Desktop Tauri IPC Provider)
 - └─ webProvider.js (Browser Worker & WASM Provider)
 - └─ webCrypto.worker.js (Изолированный Web Worker для HQC-256 & Cascade)
 - └─ vendor/liboqs-hqc256/ (Локальный vendor-independent liboqs HQC-256 Runtime)
 - └─ Frontend/frontend/src/crypto.js (Classic & PQC Primitives: noble-curves, scure-bip39, AES-GCM)
 - └─ Backend/crypto/ (Go CIRCL ML-KEM-1024, ML-DSA-87, Ed25519)
- └─ Backend Application Services (Go 1.26.5) [Backend/]
 - └─ cmd/gaiacom/main.go (CLI Entrypoint: run, setup, doctor, version)
 - └─ main.go (Server Lifecycle, Graceful Shutdown, Signal Handling)
 - └─ routes.go (HTTP Router Setup, Middleware Cascade, Route Declarations)
 - └─ route_runtime.go (Runtime Configuration Validation)
 - └─ scheduler.go (DraftScheduler: Scheduled Sending Mail/Messages)
 - └─ httpx/ (Framework-free net/http Routing, CORS & Security Headers)
 - └─ auth/ (User Accounts, Argon2/PBKDF2, JWT Signing & Rotation)
 - └─ devices/ (Device Sessions, Key Revocation, QR Pairing Protocol)
 - └─ identity/ (GaiaID Management, Public Key Catalogs, HumanProof)
 - └─ messaging/ (E2EE Message Transport, Proofs, Read Receipts, Reactions)
 - └─ mailbox/ (Mailbox State, Folders, Labels, Contacts, Filters, Search)
 - └─ room/ (Room Management, Member Roles, Join Requests, Pins, Audit Logs)
 - └─ publicchannels/ (Public Broadcasts, Subscriptions, Moderation, Comments)
 - └─ gsn/ (Social Network: Posts, Comments, Reactions, Profiles)
 - └─ storage/ (GaiaDrive Chunked Object Store: Local FS & S3 Adapter, Quotas)
 - └─ gaiaDROP/ (Anonymous Text Submissions c Rate Limiting & Hash Validation)
 - └─ smtpbridge/ (Legacy SMTP Inbound & Outbound c Downgrade Labeling)
 - └─ federation/ (S2S Signed PDUs, SSRF Firewall, Replay Protection, Queue Worker)
 - └─ noderegistry/ (Node Registration, Operator Admission, Core-Hash Integrity)
 - └─ trustmesh/ (Local Reputation Assessment, Epoch Hashing, Abuse Friction)
 - └─ governance/ (Bootstrap Roles, Reporting Cases, Gaia's Eyes Disclosure, Appeals)
 - └─ internal/security/ (GaiaShield: Cryptographic Audit Chain, Redaction Filters, IDS Rules)
 - └─ networkhealth/ (Node Metrics, Latency Pings, Dashboard Data)
 - └─ operations/ (Health Checks: /livez, /readyz, /metrics Prometheus Monitoring)
 - └─ provision/ (Idempotent Production Initialization & Environment Diagnostics)
 - └─ mobileapi/ (In-Process Gomobile Bindings для Native Android/iOS Nodes)
- └─ Persistence & Database Layer [Backend/database/ & Backend/repository/]
 - └─ database/database.go (SQLite Connection Manager, WAL Pragmas, DDL Migrations)
 - └─ database/migrations.go (Schema Migration Ledger с SHA-256 Checksums & Backups)
 - └─ database/migration_transport_queue.go (Offline Transport Outbox и Deduplication Tables)
 - └─ repository/repository.go (Central Store Interface Definitions)
 - └─ repository/sql_store.go (SQLStore Constructor и Transaction Management)
 - └─ repository/sql_store_*.go (25 Domain-Specific Persistence Adapters)
- └─ Android Mobile Platform [Android/]
 - └─ platform/contracts/ (Interface Contracts между модулями)
 - └─ platform/kernel/ (App Lifecycle, Command Bus, Event Dispatcher)
 - └─ platform/node/ (Bridge к Go Embedded Node через Backend/mobileapi)
 - └─ platform/security/ (Cryptographic Interfaces & GaiaCore Rust-JNI)
 - └─ platform/security-android/ (Android Keystore & StrongBox Hardware Binding)
 - └─ platform/security-auth/ (Biometric Authentication & PIN Lock)
 - └─ platform/runtime/ (Local Runtime Management)
 - └─ platform/remote/ (Transport Adapters для HTTP, Wi-Fi Direct, BLE)
 - └─ platform/api/ (Typed Kotlin APIs для Presentation Layer)
 - └─ platform/identity/ (Identity и Key Management на устройстве)
 - └─ app/ (Android Jetpack Compose UI Entrypoint)
- └─ Deployment, DevOps & Runtime Hardening [deploy/]
 - └─ nginx/gaiacom.conf (Hardened NGINX VHost: TLS 1.3, CSP, Frame Deny, No Sniff)
 - └─ systemd/gaiacom.service (Hardened systemd Unit: Strict Sandbox, PrivateTmp, NoNewPrivileges)
- └─ Test & Security Gates [security/ & scripts/]
 - └─ scripts/repository_hygiene.py (Hygiene Scanner: проверяет Binary Hashes, File Sizes & Secrets)
 - └─ scripts/android_architecture_gate.py (Валидирует Android Module Dependencies)
 - └─ security/master_security_suite.py (Автоматизированный End-to-End Adversarial Suite)
 - └─ security/poc/ (11 изолированных Proof-of-Concept Exploit & Integrity Tests)
 - └─ security/extreme/ (Extreme Load, BOLA/BFLA и Replay Stress Tests)

2. Привязка файлов к архитектуре

Все пути указаны относительно корня репозитория (`GaiaCom-main/`).

2.1 Frontend Client (`Frontend/frontend/`)

Entrypoints и конфигурация

- `Frontend/frontend/index.html`: корневой HTML-документ с Content Security Policy (CSP), ссылками PWA Manifest и Startup CSS.
- `Frontend/frontend/vite.config.js`: конфигурация Vite Bundler, поддержка WASM, Rollup Bundle Splitting.
- `Frontend/frontend/package.json`: определение Node Package (React 18, noble-curves, scure-bip39, vitest).
- `Frontend/frontend/src/index.jsx`: инициализация React 18 Root, `RuntimeBoundary` для изоляции ошибок, регистрация PWA Service Worker.
- `Frontend/frontend/src/index.css`: основной CSS Manifest; импортирует все 27 отдельных Stylesheets интерфейса.
- `Frontend/frontend/public/startup.css`: критический Fallback Styling до React Hydration для предотвращения FOUC.
- `Frontend/frontend/public/service-worker.js`: Offline Caching и Asset Management для Progressive Web Apps.

Глобальные компоненты и Orchestration

- `Frontend/frontend/src/App.jsx`: главный Container; управляет Theme, Identities, Menu State, Notifications и Socket/Polling Behavior.
- `Frontend/frontend/src/components/app/AppMainContent.jsx`: динамический Viewport; загружает активную Pane через `React.lazy` и `Suspense`.
- `Frontend/frontend/src/components/app/AppModalLayer.jsx`: центральный слой для Popups, Modals и Dialogs.
- `Frontend/frontend/src/components/app/AppFeedbackModals.jsx`: Alert-, Confirmation- и Three-Way Selection Dialogs.
- `Frontend/frontend/src/api.js`: центральный HTTP API Client с автоматической JWT Rotation и Desktop Detection.
- `Frontend/frontend/src/crypto.js`: Web Crypto Engine (noble/curves, scure/bip39, PBKDF2, AES-GCM, ECDSA).
- `Frontend/frontend/src/sovereignCrypto.js`: слой абстракции Sovereign Dual-PQC, переключающий Native и Web Worker Providers.

Navigation и Layout

- `Frontend/frontend/src/components/layout/NavigationSidebar.jsx`: Sidebar из трех областей (Workspace, Network, Personal), Identity Card и Quantum Shield Indicator.
- `Frontend/frontend/src/components/layout/ListPane.jsx`: средняя колонка для списков сообщений, Chat Contacts, Folders и Filters.

- `Frontend/frontend/src/components/layout/MobileNavigationDock.jsx` : Touch-оптимизированная навигация для телефонов и планшетов.
- `Frontend/frontend/src/components/layout/LogoMark.jsx` : векторный логотип GaiaCom.
- `Frontend/frontend/src/components/layout/VersionBadge.jsx` : отображение версии ПО и Federation Consensus.

Dashboard и Feature Panes

- `Frontend/frontend/src/components/chat/DashboardPane.jsx` : системный Command Center, Uplink Status и Quick Overview.
- `Frontend/frontend/src/components/chat/ChatPane.jsx` : E2EE One-to-One Quantum Chat, Timeline, UI вложений и Reactions.
- `Frontend/frontend/src/components/chat/GroupChatPane.jsx` : представление Group Room, Subchannels, Participant List и Role Display.
- `Frontend/frontend/src/components/chat/PublicChannelsPane.jsx` : Public Broadcast Channels, Posts, Reactions и Comments.
- `Frontend/frontend/src/components/chat/ReaderPane.jsx` : Mail Reader, декодирование Native Envelopes и GaiaProof Verification.
- `Frontend/frontend/src/components/chat/ComposerPane.jsx` : Mail Editor с Subject, Recipients, Draft Storage и SMTP Switch.
- `Frontend/frontend/src/components/chat/ProfilePane.jsx` : отображение Keys, Mnemonics, Inactivity Timer, удаление аккаунта и Privacy.
- `Frontend/frontend/src/components/chat/DrivePane.jsx` : GaiaDrive Storage Manager (OPFS Files, Notes, Cloud Sync).
- `Frontend/frontend/src/components/chat/DropPane.jsx` : GaiaDrop Submission Inbox и проверка анонимных обращений.
- `Frontend/frontend/src/components/chat/GsnPane.jsx` : GaiaSocialNetwork Feed, создание Posts, Follow/Unfollow.
- `Frontend/frontend/src/components/chat/SecurityCenter.jsx` : GaiaShield Security Center, Event Logs и Node Audit.
- `Frontend/frontend/src/components/chat/AbuseCenter.jsx` : Reporting Center, Trust Assessment, Reviewer Queue и Operator Actions.
- `Frontend/frontend/src/components/public/NetworkHealthDashboard.jsx` : публичный статус сети, Ping Times и Federation Health.
- `Frontend/frontend/src/components/chat/VaultPane.jsx` : *Legacy*: старый Note Vault, заменен `DrivePane.jsx`.

Authentication и Onboarding

- `Frontend/frontend/src/components/auth/AuthScreen.jsx` : экран аутентификации (Login, Registration, Recovery).
- `Frontend/frontend/src/components/auth/UnlockScreen.jsx` : локальная разблокировка Session (Password, PIN, WebAuthn PRF).
- `Frontend/frontend/src/components/auth/SetupWizard.jsx` : Wizard для генерации новых Identities и Mnemonics.

- `Frontend/frontend/src/components/onboarding/FirstRunOnboarding.jsx`: интерактивное введение после Initial Setup.

Modals и Dialogs

- `Frontend/frontend/src/components/modals/AddContactModal.jsx`: добавление контакта по GaiaID.
- `Frontend/frontend/src/components/modals/ContactProfileModal.jsx`: Contact Profile, Verification Evidence и Key History.
- `Frontend/frontend/src/components/modals/CreateGroupModal.jsx`: создание Group Room с флагом Top Secret.
- `Frontend/frontend/src/components/modals/JoinGroupModal.jsx`: вход в Group по Invitation Code.
- `Frontend/frontend/src/components/modals/CreateChannelModal.jsx`: создание Channel в Room.
- `Frontend/frontend/src/components/chat/GroupSettingsModal.jsx`: администрирование Room, Slow Mode, Role Permissions и Audit Logs.
- `Frontend/frontend/src/components/modals/KeyChangeWarningModal.jsx`: Security Warning при изменении ключа получателя.
- `Frontend/frontend/src/components/modals/QuantumShieldModal.jsx`: подробности статуса Dual-PQC Protection.
- `Frontend/frontend/src/components/common/HumanProofDialog.jsx`: создание криптографического Human Proof.
- `Frontend/frontend/src/components/common/AppModal.jsx`: универсальный Modal Container.
- `Frontend/frontend/src/components/common/GaiaPassportCard.jsx`: визуальная Passport Card со значениями Trust.
- `Frontend/frontend/src/components/common/AvatarPicker.jsx`: выбор Avatar и подготовка Upload.

Frontend Custom Hooks (`Frontend/frontend/src/hooks/`)

- `useGaiaAuth.js`: Authentication State, Token Management и получение Identity.
- `useChat.js`: Chat Messages, Polling, Sending, Decryption, Deletion и Reactions.
- `useDrive.jsx`: управление OPFS Files, локальные AES-GCM Notes и Chunked Cloud Uploads.
- `useEmails.js`: логика GaiaMail и SMTP Mailbox, Filters, Drafts, Labels и State Updates.
- `usePublicChannels.js`: подписка на Public Channels, создание Posts и управление Comments.
- `useGsn.js`: получение GSN Feeds, публикация Posts, Reactions и изменения Profile.
- `useGaiaDrop.js`: получение и очистка GaiaDrop Submissions.
- `usePresence.js`: отправка Heartbeats и запрос Online State контактов.
- `useMessageMeta.js`: Pins, Saved Messages и Reactions в Local Storage.
- `useUnreadMarkers.js`: Unread Markers и Position Tracking для Chats и Groups.
- `useChatNotifications.js`: Desktop- и Audio Notifications для новых сообщений.
- `useInactivityLock.js`: автоматическая блокировка UI при бездействии.
- `useKeyChangeDetection.js`: обнаружение попыток манипуляции ключами получателя.
- `useVault.js`: *Legacy*: старый Vault Hook, заменен `useDrive.jsx`.

Frontend Utilities (`Frontend/frontend/src/utls/`)

- `avatar.js`: генерация и масштабирование Avatars.

- `channelsTranslations.js`: многоязычные System Text для Public Channels.
- `contacts.js`: объединение Mail- и Chat Contacts.
- `deviceFanout.js`: расчет Fanout для Multi-Device Encryption.
- `gaiaAddress.js`: Parsing и Validation адресов `@user:node.tld`.
- `humanProof.js`: расчеты Proof-of-Humanity.
- `i18n.jsx`: Internationalization Engine (DE, EN, ES, FR, IT, etc.).
- `keyHistory.js`: Append-Only Tracking верифицированных Contact Keys.
- `markdown.jsx`: безопасный React Rendering Markdown без `dangerouslySetInnerHTML`.
- `notificationPreferences.js`: нормализация Notification Settings.
- `payload.js`: Validation и Serialization Message Payloads.
- `recovery.js`: Export и Import зашифрованных Recovery Files.
- `safeJson.js`: усиленный JSON Parsing против Prototype Pollution.
- `secureExport.js`: зашифрованный Export System Data.
- `terms.js`: Terms of Use и Privacy Statements.
- `useContactActions.js`: Helper Functions для управления контактами.
- `useProfileActions.js`: Helper Functions для Profile и Key Management.
- `uuid.js`: генерация и Validation RFC 4122 v4 UUID.
- `webauthnPrf.js`: WebAuthn PRF Extension для Hardware-Bound Key Derivation.

2.2 Desktop Shell (DesktopClient/)

- `DesktopClient/src-tauri/Cargo.toml`: конфигурация проекта Tauri и зависимости (`pgcrypto-hqc`, `zeroize`, `gaia-com-sovereign-core`).
- `DesktopClient/src-tauri/tauri.conf.json`: Security Boundaries: локальный Web Server отключен, строгий CSP, Window Properties.
- `DesktopClient/src-tauri/capabilities/default.json`: Fine-Grained Capability Declaration Tauri 2.
- `DesktopClient/src-tauri/src/main.rs`: Desktop Execution Entrypoint.
- `DesktopClient/src-tauri/src/lib.rs`: регистрация Native IPC Commands в Tauri Builder.
- `DesktopClient/src-tauri/src/sovereign_crypto.rs`: реализация пяти изолированных IPC Crypto Commands:
 - `sovereign_hqc256_keypair`: Native-генерация Hqc-256 Key Pair на Rust.
 - `sovereign_hqc256_encapsulate`: создание KEM Ciphertext и Shared Secret.
 - `sovereign_hqc256_decapsulate`: Decapsulation KEM Ciphertext с Private Key.
 - `sovereign_seal`: Native-выполнение Twofish/AES-GCM или четырехступенчатого Cascade.
 - `sovereign_open`: Native-расшифрование Cascade Ciphertext на Rust.

2.3 Sovereign Crypto Core & WebAssembly

- `SovereignCryptoCore/Cargo.toml`: Rust Crate `gaia-com-sovereign-core` v2.0.0.

- `SovereignCryptoCore/src/lib.rs` : математическая реализация Cipher Cascades:
 - `seal()` & `open()` : основные функции Profiles Accelerated и Top Secret.
 - Twofish-256-EAX, AES-256-GCM, Serpent-256-CTR, HMAC-SHA3-512, XChaCha20-Poly1305, AES-256-GCM-SIV.
 - Secure Zeroization (`zeroize`) всех временных Key Buffers.
 - `SovereignCryptoWasm/Cargo.toml` : Crate для WASM Compilation через `wasm-pack` .
 - `SovereignCryptoWasm/src/lib.rs` : WebAssembly Bindings (`#[wasm_bindgen]`), предоставляющие Rust Cascade браузерам.
 - `Frontend/frontend/src/sovereign/wasm/gaiacom_sovereign_wasm_bg.wasm` : скомпилированный WASM Binary.
 - `Frontend/frontend/src/sovereign/vendor/liboqs-hqc256/hqc-256.runtime.js` : локальный liboqs HQC-256 JavaScript/WASM Runtime.
 - `Frontend/frontend/src/sovereign/webCrypto.worker.js` : изолированный Web Worker, выполняющий HQC-256 Operations и Cascade в Background Thread.
-

2.4 Core Rust Engine (`Core/rust/gaiacore/`)

- `Core/rust/gaiacore/Cargo.toml` : Zero-Dependency Crate `gaiacore` .
 - `Core/rust/gaiacore/src/lib.rs` : фундаментальные Invariants:
 - `validate_gaia_id()` : RFC-compliant Validation `@user:domain.tld` .
 - `constant_time_eq()` : Timing-Resistant Byte Comparison.
 - `is_fixed_hex()` : безопасная Hexadecimal Validation.
 - `Core/rust/gaiacore/src/envelope.rs` : Parsing и Integrity Validation Message Envelopes.
 - `Core/rust/gaiacore/src/trust_mesh.rs` : математический расчет Reputation Scores и Decay Curves.
-

2.5 Backend Application Layer (`Backend/`)

Main Program и Routing

- `Backend/cmd/gaiacom/main.go` : Binary Entrypoint; делегирует в `backend.Execute()` .
- `Backend/main.go` : Server Lifecycle, инициализация Configuration, Database и Routes, Graceful Shutdown.
- `Backend/routes.go` : регистрация всех Endpoints, Middleware Pipelines и Background Workers.
- `Backend/route_runtime.go` : загрузка и Validation Environment Variables (`GAIACOM_JWT_SECRET` , etc.).
- `Backend/scheduler.go` : Background Service для Delayed/Scheduled Mail и Messages.
- `Backend/embedded_node.go` : In-Process Operation как Embedded Engine.
- `Backend/mobile_node_registry.go` : управление активными Node Instances в Mobile Contexts.
- `Backend/mobile_transport_registry.go` : регистрация доступных Mobile Transport Paths.

Domain Packages

- Backend/auth/: `auth_service.go` - User Registration, Argon2 Password Hashing, JWT Issuance; `auth_handler.go` - HTTP Handlers для `/api/v1/auth/*`.
- Backend/devices/: `device_service.go` - Device-Key Management, Revocation и QR-Code Pairing; `device_handler.go` - HTTP Handlers для `/api/v1/devices/*`.
- Backend/identity/: `identity_service.go` - CRUD Identities и публикация Public Keys; `identity_handler.go` - HTTP Handlers для `/api/v1/identity/*` и Public Profiles.
- Backend/messaging/: `messaging_service.go` - Envelope Validation, E2EE Routing и Proof Generation; `messaging_handler.go` - HTTP Handlers `/api/v1/messaging/*`.
- Backend/mailbox/: `service.go` - Mailbox Folders, Labels, Filter Rules, Address Book и Global Full-Text Search; `handler.go` - HTTP Handlers `/api/v1/mailbox/*` и `/api/v1/search/global`.
- Backend/room/: `service.go` - Group Rooms, Subchannels, Roles, Invitation Links, Join Requests; `handler.go` - `/api/v1/rooms/*`.
- Backend/publicchannels/: Public Broadcast Channels, Posts, Comments, Reactions, Moderation; Handlers `/api/v1/public-channels/*`.
- Backend/gsn/: GaiaSocialNetwork Posts, Reactions, Comments, Profiles, Followers; Handlers `/api/v1/gsn/*`.
- Backend/storage/: Chunked Upload Coordination, Quotas, ACL; File-System Object Store с Path-Jail Protection и S3/MinIO Adapter; Handlers `/api/v1/storage/*`.
- Backend/gaiadrop/: Anonymous Text Submissions, Hash Validation, Size Limiting; Handlers `/api/v1/gaiadrop/*` и `/api/v1/public/gaiadrop/*`.
- Backend/smtpbridge/: Legacy SMTP Gateway с STARTTLS, Header Sanitization и Downgrade Labeling; Handlers `/api/v1/smtp/send` и `/api/v1/public/smtp/ingest`.
- Backend/federation/: Server-to-Server Communication, Signed PDUs, Replay Protection, Federation Types и S2S Forwarding.
- Backend/noderegistry/: регистрация новых Nodes, Operator Admission, Quarantine и Core-Hash Comparison; Handlers `/api/v1/node/registry/*`.
- Backend/trustmesh/: Proof-Based Abuse Reports, Reputation Reduction, Epoch Hashing; Handler `/api/v1/reports/submit`.
- Backend/governance/: Bootstrap Operator Rights (`governance.json`), Gaia's Eyes Tokens, Reviewer Queue, Trusted Operator Bootstrap; Handlers `/api/v1/governance/*`, `/api/v1/reviewer/*`, `/api/v1/node/abuse/*`.
- Backend/internal/security/ (GaiaShield): Central Security Subsystem, EdgeShield Middleware, Retention Sweeper, Tamper-Resistant Audit Chain, Security Events, Privacy Redaction; Handlers `/api/v1/security/*` и `/api/v1/node/security/*`.
- Backend/networkhealth/: Uptime Measurement, Latency Checks, Neighboring-Node Status; Handler `/api/v1/public/network-health`.
- Backend/operations/: `monitor.go` - Liveness (`/livez`), Readiness (`/readyz`) и Prometheus Metrics (`/metrics`).
- Backend/httpx/: Zero-Dependency HTTP Router, Security Headers, CORS Policy, standardized JSON/Error Serialization.
- Backend/crypto/: CIRCL ML-KEM-1024, Ed25519 и ML-DSA-87 Helpers.
- Backend/models/: все Go Structs для Entities, API Requests, DTOs и Database Rows.
- Backend/buildinfo/: Immutable Release Metadata (Version, Git Commit, Build Time).

- Backend/provision/: `setup.go` - Idempotent Production Environment Generation; `doctor.go` - Automated Security Validation Configuration.
-

2.6 Персистентность и SQLite Database

- Backend/database/database.go: SQLite Initialization (`modernc.org/sqlite`, pure-Go, CGO-free); принудительно включает WAL (`PRAGMA journal_mode=WAL`), Foreign Keys (`PRAGMA foreign_keys=ON`), Busy Timeout (10 000 ms) и FULL Synchronous; содержит DDL Migration Table и более 70 DDL Statements.
 - Backend/database/migrations.go: Deterministic Schema Migration Manager; выполняет Atomic SQLite Backup перед каждой ожидающей Migration (`backupDatabaseBeforeMigration`).
 - Backend/database/migration_transport_queue.go: Схема для Offline Transport Outbox (`transport_outbox`) и Deduplication (`transport_inbox_dedup`).
 - Backend/repository/repository.go: Interfaces `Store`, `AuthStore`, `MessagingStore`, `StorageStore`, `GovernanceStore`, etc.
 - Backend/repository/sql_store.go: основной `SQLStore` Adapter вокруг `*sql.DB` и Transaction Helpers.
 - Backend/repository/sql_store_*.go: 25 Domain-Specific Implementation Files, включая:
 - `sql_store_auth.go`: Users, Sessions, Passwords.
 - `sql_store_devices.go` / `sql_store_device_keys.go`: Devices и Pairings.
 - `sql_store_identity.go`: Identities, Human Proofs.
 - `sql_store_messaging.go`: Envelopes, Inboxes, Proofs, Read Receipts, Reactions.
 - `sql_store_mailbox.go`: Mailbox States, Drafts, Labels, Filters, Search.
 - `sql_store_rooms_public_channels.go`: Rooms, Channels, Memberships, Moderation.
 - `sql_store_public_channel_interactions.go`: Public Channel Posts, Reactions, Comments.
 - `sql_store_gsn.go`: Posts, Comments, Reactions, Profiles, Followers.
 - `sql_store_storage_gaiadrop.go`: File Metadata, Chunks, Access Rights, Drop Submissions.
 - `sql_store_federation.go`: Server List, Queue, S2S Routing.
 - `sql_store_federation_replay.go`: Replay Checks для входящих PDUs.
 - `sql_store_governance_abuse.go`: Policies, Role Credentials, Cases, Disclosures.
 - `sql_store_security_events.go`: GaiaShield Security Events & Audit Chain.
 - `sql_store_rate_limit.go`: Rate-Limit Counters в SQLite.
 - `sql_store_presence.go`: Heartbeat и Online States.
 - `sql_store_transport_queue.go`: Multi-Hop Offline Queue.
-

2.7 Android Embedded Node & Modular Platform (Android/)

- Android/settings.gradle.kts: Gradle Multi-Module Structure:
`:app`, `:platform:contracts`, `:platform:kernel`, `:platform:node`, `:platform:security`, `:platform:security-android`, `:platform:security-auth`, `:platform:runtime`, `:platform:remote`, `:platform:api`, `:platform:identity`.

- Эти модули разделяют Native Jetpack Compose Client, Contracts, Application Kernel, JNI Bridge к Go Backend, Security Interfaces, Android Keystore/StrongBox, Biometrics, Runtime, Internet/Wi-Fi Direct/Bluetooth Transport, Typed Kotlin API Facade и Local Identity Management.
 - `Android/build.gradle.kts`: центральная Build Configuration.
-

2.8 Deployment, DevOps & Hardening (deploy/)

- `deploy/nginx/gaiacom.conf`: Hardened NGINX Reverse Proxy; пересылает `/api/v1/`, `/.well-known/gaiacom/`, `/livez`, `/readyz` на локальный Backend :8080; блокирует внешний `/metrics`; защищает Static Frontend строгими Headers `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Content-Security-Policy`.
 - `deploy/systemd/gaiacom.service`: Hardened systemd Unit для Linux AMD64 с `ProtectSystem=strict`, `ProtectHome=true`, `PrivateTmp=true`, `PrivateDevices=true`, `NoNewPrivileges=true`, `LockPersonality=true`, `RestrictSUIDSGID=true`; непривилегированный `gaiacom:gaiacom`, `UMask=0077`.
-

2.9 Security PoCs, Audits & Quality Gates (security/ & scripts/)

- `scripts/repository_hygiene.py`: предотвращает Commit Private Keys, неразрешенных Binaries, Logs и Passwords; проверяет 653+ Files.
 - `scripts/android_architecture_gate.py`: проверяет строгие Android Module Boundaries.
 - `security/master_security_suite.py`: запускает полный Automated Security Validation Suite.
 - `security/total_system_poc_runner.py`: оркестрирует System-Wide PoC Attack Scenarios.
 - `security/poc/`:
 - `poc/federation/`: SSRF Protection, IP Blocks, Invalid Signatures, Replay Attacks.
 - `poc/gaiadrop/`: Payload Size Limits, XSS Filtering, Rate Limits.
 - `poc/gaiaproof/`: Tamper Resistance GaiaProof Certificates.
 - `poc/gaiarooms-pro/`: Room Access Permissions, Role Escalation, Top Secret Integrity.
 - `poc/gaiavault/`: проверка нечитабельности client-side encrypted records на сервере.
 - `poc/governance/`: Role Credentials, Signature Chains, Threshold Consensus.
 - `poc/key-history/`: обнаружение Manipulated Recipient Keys (Downgrade Protection).
 - `poc/secure-disclosure/`: Time-Limited Disclosure через Gaia's Eyes Tokens.
 - `poc/smtp-bridge/`: защита от CRLF Injection и Strict Downgrade Signaling.
 - `poc/total-system/`: Full-System Hardening Test.
 - `poc/trust-passport/`: Validation Reputation Metrics против Forgery.
 - `security/extreme/`: Extreme Stress Tests для Concurrent Database Writes и BOLA/BFLA Attacks.
-

3. Документация модулей

Модуль 1: Sovereign Cryptographic Engine (Dual-PQC & Multi-Cipher Cascade)

Назначение

Фундаментальная криптографическая основа GaiaCom. Обеспечивает конфиденциальность, аутентичность и постквантовую защиту сообщений, файлов и идентичностей.

Возможности

- **Dual-PQC Hybrid KEM:** объединяет Ephemeral X25519, ML-KEM-1024 (FIPS 203) и HQC-256 (NIST PQC Round 4) через HKDF-SHA3-512 в общий Root Secret.
- **Sovereign Accelerated Cascade:** аутентифицированное шифрование Twofish-256-EAX + AES-256-GCM с подписью Ed25519.
- **Sovereign Top Secret Cascade:** четырехступенчатый каскад Serpent-256-CTR-HMAC-SHA3-512 + Twofish-256-EAX + XChaCha20-Poly1305 + AES-256-GCM-SIV с двойной подписью Ed25519 и ML-DSA-87 (FIPS 204).
- **Transcript Binding:** криптографическая привязка Version, Sender, Recipient, Message ID, Timestamp и Algorithm Suite к Ciphertext через AAD.
- **Fail-Closed Provider Architecture:** переключение между Tauri Native Host и изолированным Web Worker; немедленная блокировка при отсутствии необходимых Primitives.

Связанные файлы

```
Frontend:
- Frontend/frontend/src/sovereignCrypto.js
- Frontend/frontend/src/crypto.js
- Frontend/frontend/src/sovereign/webProvider.js
- Frontend/frontend/src/sovereign/nativeProvider.js
- Frontend/frontend/src/sovereign/webCrypto.worker.js
- Frontend/frontend/src/sovereign/wasm/gaiacom_sovereign_wasm.js
- Frontend/frontend/src/sovereign/vendor/liboqs-hqc256/hqc-256.runtime.js

Desktop Shell:
- DesktopClient/src-tauri/src/sovereign_crypto.rs
- DesktopClient/src-tauri/src/lib.rs

Sovereign Core (Rust):
- SovereignCryptoCore/src/lib.rs
- SovereignCryptoWasm/src/lib.rs

Core Primitives (Rust):
- Core/rust/gaiacore/src/lib.rs
- Core/rust/gaiacore/src/envelope.rs

Backend:
- Backend/crypto/crypto_service.go
- Backend/crypto/kyber.go
- Backend/crypto/types/crypto_types.go

Tests:
- Frontend/frontend/src/adversarial.test.js
- Frontend/frontend/src/hqc-smoke.js
- SovereignCryptoCore/tests / DesktopClient/src-tauri/tests
```

Интеграция с Dashboard

- Используется в `ChatPane.jsx`, `GroupChatPane.jsx`, `ComposerPane.jsx`, `ReaderPane.jsx`, `DrivePane.jsx`.
- Визуальные индикаторы: `QuantumShieldModal.jsx`, `Badge HQC-256 + ML-KEM-1024 VERIFIED`, `Security Status` в `NavigationSidebar.jsx`.
- Пользователь может переключать *Accelerated / Top Secret* и просматривать `Cryptographic Verification Certificate`.

Зависимости

```
Sovereign Cryptographic Engine
├─ требует WebAssembly & Web Worker runtime в браузере
├─ требует pqcrypto-hqc & gaiacom-sovereign-core в Desktop Client
├─ требует cloudflare/circl в Backend (ML-KEM / ML-DSA)
└─ экспортирует Primitives в Messaging, Storage, GSN и Drive
```

Используется модулями

Module 2, Module 4, Module 5, Module 6, Module 9, Module 10.

Модуль 2: Authentication, Sessions & Key Vault (GaiaVault / Auth)

Назначение

Управляет User Accounts, безопасной Client-Side генерацией и хранением ключей, Session Tokens (JWT Access/Refresh) и Pairing новых устройств по QR Code.

Возможности

- Account Registration с Argon2id Password Hashing на сервере и PBKDF2/BIP-39 Key Derivation на клиенте.
- Short-Lived JWT Access Tokens и Long-Lived Refresh Tokens с автоматической Rotation.
- Device Pairing через Ephemeral One-Time Secrets без передачи Private Keys.
- Inactivity Lock с локальной разблокировкой Password, PIN или WebAuthn PRF Hardware Token.

Связанные файлы

```
Frontend:
- Frontend/frontend/src/components/auth/AuthScreen.jsx
- Frontend/frontend/src/components/auth/UnlockScreen.jsx
- Frontend/frontend/src/components/auth/SetupWizard.jsx
- Frontend/frontend/src/hooks/useGaiaAuth.js
- Frontend/frontend/src/hooks/useInactivityLock.js
- Frontend/frontend/src/utls/webauthnPrf.js
- Frontend/frontend/src/utls/recovery.js

Backend:
- Backend/auth/auth_service.go
- Backend/auth/auth_handler.go
- Backend/devices/device_service.go
- Backend/devices/device_handler.go

API:
```

```
- POST /api/v1/auth/register, /login, /refresh, /logout, /change-password, /delete-account
- GET /api/v1/auth/status, /devices
- POST /api/v1/devices/pairings, /devices/pairings/:id/approve, /consume
```

Database:

```
- users, identities, device_sessions, device_pairings, device_keys
```

Dashboard / зависимости

После Login открывается `DashboardPane.jsx`; `UnlockScreen.jsx` разблокирует сессию после Inactivity; `ProfilePane.jsx` управляет активными устройствами. Модуль использует Module 1 и Shared Components `api.js`, `safeJson.js`; является prerequisite для всех остальных модулей.

Модуль 3: Identity, GaiaID & Trust Passport

Назначение

Обеспечивает децентрализованную адресацию `@username:node.domain.tld`, криптографические доказательства идентичности и Trust Passport.

Возможности

- Parsing/Validation федеративных GaiaID.
- Public Key Catalogs: X25519, ML-KEM-1024, HQC-256, Ed25519, ML-DSA-87.
- HumanProof Verification.
- Trust Passport с агрегированными Trust/Reputation Metrics.
- Key Change Detection для защиты от MITM/Key Substitution.

Связанные файлы

Frontend:

```
- Frontend/frontend/src/components/common/GaiaPassportCard.jsx
- Frontend/frontend/src/components/common/HumanProofDialog.jsx
- Frontend/frontend/src/components/modals/ContactProfileModal.jsx
- Frontend/frontend/src/components/modals/KeyChangeWarningModal.jsx
- Frontend/frontend/src/hooks/useKeyChangeDetection.js
- Frontend/frontend/src/utils/gaiaAddress.js
- Frontend/frontend/src/utils/humanProof.js
- Frontend/frontend/src/utils/keyHistory.js
```

Backend:

```
- Backend/identity/identity_service.go
- Backend/identity/identity_handler.go
```

API:

```
- POST /api/v1/identity/create, /human-proof
- GET /api/v1/identity/me, /public/identity/:gaiaID, /public/trust-passport/:gaiaID
```

Database:

```
- identities, user_notification_preferences
```

Активная Identity показывается в `NavigationSidebar.jsx` и `DashboardPane.jsx`; детали контакта - в `ContactProfileModal.jsx`. Используется коммуникационными модулями и Governance.

Модуль 4: Quantum Chat & E2EE Direct Messaging

Назначение

Прямая 1-to-1 E2EE-коммуникация с Dual-PQC Key Exchange, Read Receipts, Real-Time Presence и Reactions.

Возможности

- Постквантово защищенное шифрование каждого сообщения на устройстве отправителя.
- Сервер хранит только нечитаемые `message_envelopes` и Delivery Evidence.
- Уровни *Accelerated* и *Top Secret*.
- Online/Typing Presence.
- Edit, Delete, Reactions.
- GaiaProof Certificates для Abuse Evidence.

Связанные файлы

```
Frontend:
- Frontend/frontend/src/components/chat/ChatPane.jsx
- Frontend/frontend/src/components/chat/MessageActions.jsx
- Frontend/frontend/src/hooks/useChat.js
- Frontend/frontend/src/hooks/usePresence.js
- Frontend/frontend/src/hooks/useMessageMeta.js
- Frontend/frontend/src/hooks/useUnreadMarkers.js
Backend:
- Backend/messaging/messaging_service.go
- Backend/messaging/messaging_handler.go
- Backend/presence/service.go
- Backend/presence/handler.go
API:
- POST /api/v1/messaging/send, /read, /edit, /reaction, /delete, /clear
- GET /api/v1/messaging/inbox, /proof
- POST /api/v1/presence/heartbeat, /typing; GET /presence/status, /typing
Database:
- message_envelopes, inboxes, message_proofs, delivery_receipts, message_read_receipts, message_reactions, identity_presence
```

Зависит от Sovereign Crypto, Identity и GaiaShield; интегрируется с GaiaDrive для File Sharing.

Модуль 5: Group Chats, Rooms & Channels

Назначение

Multi-User Rooms с Role-Based Access Control, Subchannels, Join Requests, Invitation Links и Moderation Logs.

Возможности

- Topic-oriented Channels внутри Rooms.
- Role Hierarchy: `owner`, `admin`, `moderator`, `member`.
- Private/Public Rooms с `room_join_requests`.
- Time-/Usage-Limited Invitation Links.
- Pinned Messages и Slow Mode.

- Immutable Revision/Moderation Logs.

Связанные файлы

```
Frontend:
- Frontend/frontend/src/components/chat/GroupChatPane.jsx
- Frontend/frontend/src/components/chat/GroupSettingsModal.jsx
- Frontend/frontend/src/components/modals/CreateGroupModal.jsx
- Frontend/frontend/src/components/modals/JoinGroupModal.jsx
- Frontend/frontend/src/components/modals/CreateChannelModal.jsx
Backend:
- Backend/room/service.go
- Backend/room/handler.go
API:
- POST /api/v1/rooms/create, /update, /join, /leave, /delete, /channels, /members/role, /members/kick, /transfer-ownership
- POST /api/v1/rooms/pins/toggle, /invites/create, /invites/join, /join-requests/create, /join-requests/moderate
- GET /api/v1/rooms, /channels, /search, /pins, /join-requests, /moderation-logs
Database:
- rooms, room_members, channels, room_pinned_messages, room_invite_links, room_join_requests, room_moderation_logs
```

Использует Messaging Envelopes и GaiaShield для Moderation Actions.

Модуль 6: GaiaMail & Mailbox State Management

Назначение

Полноценная Client-Side-Encrypted Mail System с Folders, Labels, Server-Side Scheduling и Global Search.

Возможности

Inbox, Drafts, Sent, Starred, Important, Snoozed, Archive, Spam, Trash; Custom Labels; `DraftScheduler`; Filter Rules по Sender/Subject; privacy-conscious Global Search.

Связанные файлы

```
Frontend:
- Frontend/frontend/src/components/layout/ListPane.jsx
- Frontend/frontend/src/components/chat/ReaderPane.jsx
- Frontend/frontend/src/components/chat/ComposerPane.jsx
- Frontend/frontend/src/hooks/useEmails.js
Backend:
- Backend/mailbox/service.go
- Backend/mailbox/handler.go
- Backend/scheduler.go
API:
- GET /api/v1/mailbox/messages, /drafts, /labels, /contacts, /filters, /settings, /search/global
- POST /api/v1/mailbox/state, /drafts/save, /drafts/delete, /labels/save, /contacts/save, /filters/save, /settings
Database:
- mailbox_states, mail_drafts, mail_labels, mail_contacts, mail_filter_rules, mail_settings
```

Открывается через GaiaMail Navigation; непочитанные сообщения отображаются в Dashboard.

Модуль 7: SMTP Legacy Bridge & Downgrade Gateway

Назначение

Обмен с традиционными Email Systems по SMTP с явным предупреждением о потере Native GaiaCom Security Guarantees.

Возможности

- Outbound Email через STARTTLS с защитой от CRLF/Header Injection.
- Inbound Webhook `/api/v1/public/smtp/ingest` с Token Authentication.
- SMTP Messages помечаются `untrusted` во Frontend.
- Persistent Warning в Composer при SMTP Mode.

Связанные файлы

```
Frontend:  
- Frontend/frontend/src/components/layout/NavigationSidebar.jsx  
- Frontend/frontend/src/components/chat/ComposerPane.jsx  
Backend:  
- Backend/smtpbridge/service.go  
- Backend/smtpbridge/handler.go  
API:  
- POST /api/v1/smtp/send  
- POST /api/v1/public/smtp/ingest
```

Модуль 8: Public Channels

Назначение

Открытые подписные информационные Channels для Announcements, Discussion Threads, Media и Community Updates.

Возможности

Создание Channels с Name/Description/Avatar/Categories; Subscribe/Unsubscribe; Posts с Formatting/Media/Scheduling; Emoji Reactions и Comment Threads; Blocking и Discovery Search.

Связанные файлы

```
Frontend:  
- Frontend/frontend/src/components/chat/PublicChannelsPane.jsx  
- Frontend/frontend/src/hooks/usePublicChannels.js  
Backend:  
- Backend/publicchannels/service.go  
- Backend/publicchannels/handler.go  
API:  
- GET /api/v1/public-channels, /posts, /discover  
- POST /api/v1/public-channels/create, /update, /comments, /delete, /subscribe, /unsubscribe, /block, /unblock  
- POST /api/v1/public-channels/posts/create, /reaction, /comment, /pin, /comments/delete, /comments/moderate  
Database:  
- public_channels, public_channel_admins, public_channel_subscribers, public_channel_posts, public_channel_post_reactions,  
public_channel_post_comments, public_channel_blocks
```

Модуль 9: GSN - GaiaSocialNetwork (Decentralized Social Layer)

Назначение

Децентрализованная Federation-Capable Social Network для Microblogging, Status Updates и Community Interactions.

Возможности

Cryptographically Signed Posts, Node/Following Feeds, Signed Comments, Emoji Reactions, Profiles с Verified Badges и Client-Side Decryption Avatars/Post Images.

Связанные файлы

```
Frontend:
- Frontend/frontend/src/components/chat/GsnPane.jsx
- Frontend/frontend/src/components/chat/gsn/GsnPostCard.jsx
- Frontend/frontend/src/components/chat/gsn/GsnPostComposer.jsx
- Frontend/frontend/src/components/chat/gsn/GsnProfileEditor.jsx
- Frontend/frontend/src/components/chat/gsn/DecryptedAvatar.jsx
- Frontend/frontend/src/components/chat/gsn/DecryptedGsnImage.jsx
- Frontend/frontend/src/hooks/useGsn.js
Backend:
- Backend/gsn/service.go
- Backend/gsn/handler.go
API:
- POST /api/v1/gsn/posts, /posts/:id/react, /posts/:id/comment, /follow, /unfollow, /profile
- GET /api/v1/gsn/feed/node, /feed/following, /posts/:id/comments, /profile/:gaia_id
- DELETE /api/v1/gsn/posts/:id, /posts/:id/comments/:commentId
Database:
- gsn_posts, gsn_post_comments, gsn_post_reactions, gsn_follows, gsn_profiles
```

Модуль 10: GaiaDrive & Encrypted Chunk Storage

Назначение

Объединяет локальное зашифрованное OPFS-хранилище с опциональной E2EE Cloud Synchronization через Backend.

Возможности

AES-GCM Local Notes/Files; 1 MB Chunk Upload с SHA-256; User Quotas; 14-Day Cloud TTL/Retention Sweeper; Local Path-Jail или S3-Compatible Object Store.

Связанные файлы

```
Frontend:
- Frontend/frontend/src/components/chat/DrivePane.jsx
- Frontend/frontend/src/hooks/useDrive.jsx
Backend:
- Backend/storage/storage_service.go
- Backend/storage/storage_handler.go
- Backend/storage/object_store.go
- Backend/storage/s3_object_store.go
API:
- POST /api/v1/storage/init, /chunk, /complete, /grant
- GET /api/v1/storage/download/:fileId
```

Database:

- file_metadata, file_access_grants, file_chunks

Модуль 11: GaiaDrop (Anonymous Text Submissions)

Назначение

Позволяет внешним или неаутентифицированным лицам безопасно отправлять конфиденциальные Text Messages на GaiaID как Whistleblower/Drop Channel.

Возможности

Public Submit Endpoint, 64 KB Limit, XSS Sanitization, IP-Based Rate Limiting и защищенное управление в `DropPane.jsx`.

Связанные файлы

```
Frontend:
- Frontend/frontend/src/components/chat/DropPane.jsx
- Frontend/frontend/src/hooks/useGaiaDrop.js
Backend:
- Backend/gaiadrop/service.go
- Backend/gaiadrop/handler.go
API:
- POST /api/v1/public/gaiadrop/submit
- GET /api/v1/gaiadrop/inbox
- POST /api/v1/gaiadrop/read, /delete
Database:
- gaia_drop_submissions
```

Модуль 12: Federation & S2S PDU Transport

Назначение

Соединяет независимые GaiaCom Servers в децентрализованную сеть через Cryptographically Signed PDUs.

Возможности

- S2S Authorization через `X-Gaia-S2S-V1` Header.
- SSRF/DNS-Rebinding Firewall для localhost/private/loopback ranges.
- Replay Protection через `federation_replay_guard`.
- Асинхронная `federation_queues` с Exponential Backoff и Lease Locking.

Связанные файлы

```
Frontend:
- Frontend/frontend/src/components/public/NetworkHealthDashboard.jsx
Backend:
- Backend/federation/federation_service.go
- Backend/federation/federation_handler.go
- Backend/federation/federation_types.go
API:
- GET /.well-known/gaiacom/server, /nodeinfo, /nodes
```

```
- POST /.well-known/gaiacom/s2s/v1/forward, /_gaiacom/s2s/v1/forward
- GET /api/v1/public/nodes
Database:
- federation_servers, federation_queues, federation_replay_guard
```

Модуль 13: Node Registry & Core-Hash Verification

Назначение

Контролируемый Onboarding новых Server Nodes, обнаружение Software Forks и сохранение Version Compatibility сети.

Возможности

Public Ping Registration; Operator Workflow `pending` -> `accepted` / `quarantined` / `blocked`; сравнение `GAIACOM_CORE_HASH` для обнаружения измененных/устаревших Core Builds.

Связанные файлы

```
Frontend:
- Frontend/frontend/src/components/chat/SecurityCenter.jsx (Nodesystem tab)
Backend:
- Backend/noderegistry/service.go
- Backend/noderegistry/handler.go
API:
- POST /api/v1/public/node-registry/ping
- GET /api/v1/public/node-registry/nodes
- GET /api/v1/node/registry/summary
- POST /api/v1/node/registry/secrets, /ping-main, /:domain/status
Database:
- node_registry_entries
```

Модуль 14: TrustMesh & Local Reputation Scoring

Назначение

Локальное Proof-Based обнаружение и Throttling Spam/Malicious Behavior без нарушения E2EE.

Возможности

Cryptographic Abuse Reports, локальные Reputation Deductions (`abuse_scores`) с Decay Curves и Progressive Friction вплоть до Quarantine.

Связанные файлы

```
Frontend:
- Frontend/frontend/src/components/chat/ReaderPane.jsx
Backend:
- Backend/trustmesh/service.go
- Backend/trustmesh/handler.go
- Core/rust/gaiacore/src/trust_mesh.rs
API:
- POST /api/v1/reports/submit
```

```
Database:
- reports, abuse_scores
```

Модуль 15: Governance, Gaia's Eyes & Reviewer Workflows

Назначение

Role-Based управление Incidents/Abuse Reports через Reviewer Panels, Limited Case Disclosure и Transparency Reports.

Возможности

Threshold Consensus; Bootstrap Anchoring через `governance.json` и `role_credentials`; Single-Use Time/Scope-Limited Gaia's Eyes Tokens; Appeals; Tamper-Resistant Transparency Snapshots.

Связанные файлы

```
Frontend:
- Frontend/frontend/src/components/chat/AbuseCenter.jsx
Backend:
- Backend/governance/service.go
- Backend/governance/handler.go
- Backend/governance/bootstrap.go
API:
- GET /api/v1/governance/roles, /reports/mine, /reports/:caseID, /reviewer/cases, /node/abuse/queue, /public/transparency
- POST /api/v1/reports, /reports/:caseID/disclosures, /reports/:caseID/appeal, /gaia-eyes/grants, /reviewer/cases/:caseID/review, /node/abuse/actions, /node/transparency/snapshot, /public/gaia-eyes/consume
Database:
- governance_policies, role_credentials, role_credential_revocations, abuse_cases, abuse_case_events, abuse_disclosure_packages, abuse_disclosure_requests, gaia_eyes_grants, abuse_reviews, abuse_actions, abuse_appeals, federation_abuse_signals, transparency_snapshots
```

Модуль 16: GaiaShield Security Operations & Intrusion Defense

Назначение

Внутренняя Security/Monitoring System для защиты целостности узла, противодействия атакам и Tamper-Resistant Auditing.

Возможности

EdgeShield Pre-Routing Inspection; Cryptographic Audit Chain; Privacy-Preserving Redaction; IDS Rules и Automatic Quarantine.

Связанные файлы

```
Frontend:
- Frontend/frontend/src/components/chat/SecurityCenter.jsx
Backend:
- Backend/internal/security/shield.go
- Backend/internal/security/audit.go
- Backend/internal/security/events.go
- Backend/internal/security/privacy.go
- Backend/internal/security/security_routes.go
```

```
- Backend/internal/security/*_guard.go
API:
- GET /api/v1/public/security/health, /security/me/summary, /security/me/events, /security/me/report, /node/security/summary, /
node/security/events
- POST /api/v1/security/me/events/:event_id/acknowledge
Database:
- security_events, security_event_private_context, security_rules, security_rule_hits, security_user_acknowledgements,
security_quarantines, security_audit_chain, security_rate_limits
- Triggers: trg_security_events_immutable_update, trg_security_events_no_delete, trg_security_audit_chain_no_update,
trg_security_audit_chain_no_delete
```

Модуль 17: Desktop Client Integration (Tauri 2 Bridge)

Назначение

Native Desktop Runtime для Windows, macOS и Linux с изоляцией криптографических операций в Rust Process.

Возможности

- Нет Local HTTP Port: связь исключительно через Native Tauri IPC.
- HQC-256 и Multi-Cipher Cascades выполняются как Compiled Machine Code без WASM Overhead.
- Strict CSP и запрет `unsafe-eval`.

Связанные файлы

```
Frontend:
- Frontend/frontend/src/sovereign/nativeProvider.js
Desktop Shell:
- DesktopClient/src-tauri/Cargo.toml
- DesktopClient/src-tauri/tauri.conf.json
- DesktopClient/src-tauri/src/lib.rs
- DesktopClient/src-tauri/src/sovereign_crypto.rs
```

Модуль 18: Android Mobile Platform (Embedded Node)

Назначение

Работа как полноценный Mobile Communication Node с Hardware-Backed Security и Offline Capability.

Возможности

- In-Process Go Backend без открытого Network Port (`Backend/mobileapi/node.go`).
- Модульная Android Architecture с разделением Compose, Kernel, Security и Transport.
- Hardware Binding к Android Keystore и StrongBox Keymaster.
- Offline Multi-Hop Transport через Internet, Wi-Fi Direct и Bluetooth Low Energy.

Связанные файлы

Android:

- Android/settings.gradle.kts
- Android/platform/contracts/
- Android/platform/kernel/
- Android/platform/node/
- Android/platform/security-android/
- Android/platform/security-auth/
- Android/app/

Backend In-Memory Bridge:

- Backend/mobileapi/node.go
 - Backend/mobileapi/bootstrap.go
 - Backend/mobileapi/identity.go
 - Backend/mobileapi/transport.go
-

4. Dashboard & Workspace - подробная документация

Все представления Dashboard и Workspace централизованно переключаются во Frontend изменением состояния `currentMenu` в `Frontend/frontend/src/App.jsx` и `Renderer AppMainContent.jsx`.

Dashboard -> Command Center (/dashboard)

- **Internal Menu State:** `currentMenu === 'dashboard'`
- **Frontend File:** `Frontend/frontend/src/components/chat/DashboardPane.jsx`
- **CSS / Styling:** `dashboard.css`, `nebula.css`, `quantum-interface.css`
- **Компоненты:** `LogoMark`, `VersionBadge`, Metric Cards, Quick-Access Grid, PWA Install Banner.
- **Модули:** Module 1, Module 2, Module 4, Module 5.
- **API:** без прямых HTTP Calls при Rendering; используется синхронизированный State `useEmails`, `useChat`, `useGaiaAuth` и `/api/v1/auth/status`.
- **Backend:** `auth.GetStatus`, `operations.Liveness`; Services `auth.AuthService`, `operations.Monitor`.
- **Database:** `users`, `identities`, `device_sessions`.
- **Auth:** аутентифицированный пользователь с валидным JWT.

```
Пользователь открывает Dashboard
↓
App.jsx инициализирует DashboardPane.jsx
↓
Агрегируется State (Identity, unread mail, active rooms, chat metrics)
↓
Отображаются Uplink Status, PQC Security Level и Quick Actions
↓
Выбор действия переключает currentMenu
```

Dashboard -> Quantum Chat (/chat)

- **State:** `currentMenu === 'chat'`
- **Frontend:** `ChatPane.jsx`; Styles `chat.css`, `responsive.css`, `nebula.css`.
- **Компоненты:** `MessageActions.jsx`, Emoji Picker, File-Upload Indicator, TopSecret Toggle.
- **API:** POST `/api/v1/messaging/send`, `/read`, `/reaction`, `/delete`; GET `/api/v1/messaging/inbox`; Presence `/api/v1/presence/heartbeat`, `/status`.
- **Backend/Services:** `messaging.MessagingHandler.*`, `presence.PresenceHandler.*`, `messaging.MessagingService`, `presence.PresenceService`, `security.SecuritySystem`.
- **Database:** `message_envelopes`, `inboxes`, `message_proofs`, `message_reactions`, `identity_presence`.
- **Auth:** JWT + проверка владения Identity (`senderIdentityId`).

```
Пользователь отправляет сообщение
↓
ChatPane.jsx / useChat.js
↓
```

```

sovereignCrypto.js (Dual-PQC KEM + Twofish/AES-GCM или 4-stage cascade)
↓
POST /api/v1/messaging/send (только Ciphertext Envelope)
↓
MessagingHandler.SendMessage() -> MessagingService.SaveAndRouteEnvelope()
↓
SQLStore: INSERT message_envelopes & inboxes
↓
Recipient Client: GET /api/v1/messaging/inbox
↓
Recipient Worker выполняет Client-Side Decryption HQC/ML-KEM
↓
Отображение в ChatPane

```

Dashboard -> GaiaMail Inbox & Folders (/inbox, etc.)

- **State:** inbox | sent | drafts | starred | important | archive | trash.
- **Frontend:** ListPane.jsx, ReaderPane.jsx; Styles listpane.css, reader.css, product-refresh.css.
- **API:** GET /api/v1/mailbox/messages?identityId=...&folder=..., POST /api/v1/mailbox/state, GET /api/v1/messaging/proof?messageId=...
- **Backend:** mailbox.Handler.ListMessages, UpdateStates, messaging.MessagingHandler.GetMessageProof.
- **Database:** mailbox_states, message_envelopes, message_proofs.
- **Auth:** JWT Bearer Token.

```

Пользователь выбирает Folder
↓
ListPane.jsx -> getMailboxMessages()
↓
GET /api/v1/mailbox/messages
↓
mailbox.Handler -> mailbox.Service -> SQLStore
↓
Client получает Envelopes и расшифровывает Subject/Body в Web Worker
↓
ReaderPane.jsx проверяет Signature и показывает GaiaProof Seal

```

Dashboard -> SMTP Mailbox (/smtp_inbox, etc.)

- **State:** smtp_inbox | smtp_sent | smtp_trash |
- **Frontend:** ListPane.jsx, ReaderPane.jsx; Styles listpane.css, reader.css, gaiadrop.css.
- **Компоненты:** желтые Legacy Transport Warning Banners, Sender Verification.
- **API:** GET /api/v1/mailbox/messages?untrusted=true.
- **Backend/Services:** mailbox.Handler.ListMessages, mailbox.Service, smtpbridge.Service.
- **Database:** mailbox_states, message_envelopes.

```

External Mail -> SMTP Gateway
↓
POST /api/v1/public/smtp/ingest (Header Checks, SPF/DKIM)
↓
SMTPBridgeService сохраняет Envelope с untrusted = 1
↓
Client получает SMTP List

```

Dashboard -> Group Chats & Rooms (/groups)

- **Frontend:** GroupChatPane.jsx; Styles groupchat.css, chat.css.
- **Компоненты:** Channel List, Participant Bar, Role Manager, Slow Mode Timer, Pin Manager.
- **API:** GET /api/v1/rooms, /rooms/channels, /rooms/pins; POST /rooms/channels, /rooms/members/role, /rooms/pins/toggle; POST /api/v1/messaging/send.
- **Backend:** room.Handler.*, messaging.MessagingHandler.SendMessage.
- **Database:** rooms, room_members, channels, room_pinned_messages, message_envelopes.
- **Auth:** JWT + Room Membership/Role Checks.

```
Выбор Room/Channel
↓
GroupChatPane.jsx получает Channel Messages
↓
Client-Side Decryption Group Envelopes
↓
POST /api/v1/messaging/send (channel_id)
↓
Backend проверяет Write Permission & Slow Mode
↓
Сохранение message_envelopes + Fanout в Room Inboxes
```

Dashboard -> Public Channels (/public_channels)

- **Frontend:** PublicChannelsPane.jsx; Style publicChannels.css.
- **Компоненты:** Feed Cards, Post Editor, Comment Threads, Discovery Dialog.
- **API:** GET /api/v1/public-channels, /posts, /discover; POST /posts/create, /posts/reaction, /posts/comment.
- **Backend/Service:** publicchannels.Handler.*, publicchannels.Service.
- **Database:** public_channels, public_channel_posts, public_channel_post_reactions, public_channel_post_comments.
- **Auth:** JWT.

Dashboard -> GSN Social Network (/gsn)

- **Frontend:** GsnPane.jsx; Style gsn.css.
- **Компоненты:** GsnPostCard, GsnPostComposer, GsnProfileEditor, DecryptedAvatar, DecryptedGsnImage.
- **API:** GET /api/v1/gsn/feed/node, /feed/following, /profile/:gaia_id; POST /api/v1/gsn/posts, /posts/:id/react, /posts/:id/comment, /follow.
- **Services:** gsn.Service, federation.Service.
- **Database:** gsn_posts, gsn_post_comments, gsn_post_reactions, gsn_follows, gsn_profiles.
- **Auth:** JWT + Ed25519 Post Signature.

Dashboard -> Network Health (/network_health)

- **Frontend:** NetworkHealthDashboard.jsx; Style networkHealth.css.
 - **Компоненты:** Status Tiles, Latency Charts, Federation Table.
 - **API:** GET /api/v1/public/network-health, GET /api/v1/public/version.
 - **Backend/Service:** networkhealth.Handler.Dashboard, networkhealth.Service.
 - **Database:** federation_servers, node_registry_entries.
 - **Auth:** публичный доступ без Token.
-

Dashboard -> Security Center (/security_center)

- **Frontend:** SecurityCenter.jsx; Style security.css.
 - **Компоненты:** Audit Log Table, Event Acknowledgment, Nodesystem Monitor, Secret Generator.
 - **API:** GET /api/v1/security/me/summary, /events, /report; POST /api/v1/security/me/events/:event_id/acknowledge; Operator Paths /api/v1/node/security/*, /node/registry/summary.
 - **Services:** security.SecuritySystem, noderegistry.Service.
 - **Database:** security_events, security_audit_chain, node_registry_entries.
 - **Auth:** JWT; Operator Rights требуют governance.json Verification.
-

Dashboard -> Abuse Center (/abuse_center)

- **Frontend:** AbuseCenter.jsx; Style abuseCenter.css.
 - **Компоненты:** Incident Queue, Case Details, Evidence Decryptor, Threshold Voting.
 - **API:** GET /api/v1/governance/roles, /reports/mine, /reviewer/cases, /node/abuse/queue; POST /api/v1/reports, /reviewer/cases/:caseID/review, /node/abuse/actions.
 - **Backend/Service:** governance.Handler.*, governance.Service.
 - **Database:** abuse_cases, abuse_reviews, abuse_actions, role_credentials.
 - **Auth:** JWT + Reviewer/Operator Role Check.
-

Dashboard -> Address Book / Contacts (/contacts)

- **Frontend:** ListPane.jsx, ContactProfileModal.jsx; Styles listpane.css, modals.css.
 - **Компоненты:** Contact Cards, Trust Badge, Key Fingerprint, Notes.
 - **API:** GET /api/v1/mailbox/contacts, POST /api/v1/mailbox/contacts/save, GET /api/v1/public/identity/:gaiaID.
 - **Services:** mailbox.Service, identity.IdentityService.
 - **Database:** mail_contacts, identities.
-

Dashboard -> My Profile & Security (/profile)

- **Frontend:** ProfilePane.jsx; Styles profile.css, auth.css.
- **Компоненты:** Password Change Form, Mnemonic Reveal Modal, PIN/WebAuthn Switch, Session List.

- **API:** `POST /api/v1/auth/change-password`, `/privacy`, `/delete-account`; `GET /api/v1/auth/devices`.
 - **Backend/Service:** `auth.AuthHandler.*`, `auth.AuthService`.
 - **Database:** `users`, `identities`, `device_sessions`.
-

Dashboard -> GaiaDrive / Vault (`/vault`)

- **Frontend:** `DrivePane.jsx`; Style `drive.css`.
 - **Компоненты:** OPFS File Manager, Note Editor, Cloud-Sync Status, Upload Progress.
 - **API:** `POST /api/v1/storage/init`, `/chunk`, `/complete`; `GET /api/v1/storage/download/:fileId`.
 - **Backend/Services:** `storage.StorageHandler.*`, `storage.Service`, `object_store.*`.
 - **Database:** `file_metadata`, `file_chunks`, `file_access_grants`.
-

Dashboard -> GaiaDrop (`/gaiadrop`)

- **Frontend:** `DropPane.jsx`; Style `gaiadrop.css`.
 - **Компоненты:** Submission List, Integrity Hash Verifier, Detail Viewer.
 - **API:** `GET /api/v1/gaiadrop/inbox`, `POST /api/v1/gaiadrop/read`, `/delete`.
 - **Backend/Service:** `gaiadrop.Handler.*`, `gaiadrop.Service`.
 - **Database:** `gaia_drop_submissions`.
-

Dashboard -> Mail Composer Overlay (`isComposing`)

- **State:** `isComposing === true`.
 - **Frontend:** `ComposerPane.jsx`; Styles `reader.css`, `ui.css`.
 - **Компоненты:** Rich-Text Editor, Recipient Autocomplete, SMTP Toggle, Attachment Preview.
 - **API:** `POST /api/v1/messaging/send`, `POST /api/v1/smtp/send`, `POST /api/v1/mailbox/drafts/save`.
 - **Backend:** `messaging.MessagingHandler.SendMessage`, `smtpbridge.Handler.Send`, `mailbox.Handler.SaveDraft`.
-

5. CSS- и UI-архитектура

Архитектура стилей GaiaCom основана на полностью модульном CSS-каскаде без внешних CSS-фреймворков вроде Tailwind или Bootstrap. Все стили собираются в `Frontend/frontend/src/index.css` через `@import` в строго контролируемом порядке.

5.1 Глобальные стили и Reset

- `base.css`: CSS-reset, глобальные правила box-sizing, базовая типографика (Inter / Roboto), стилизация scrollbar (`gaia-scrollbar`).
- `startup.css`: предварительно загружаемые стили в HTML header; отображают fault screen (`astraea-startup--fault`) и loading states без FOUC.

5.2 Theme и Design Tokens (Dark / Light Mode)

- `theme.css`: определяет CSS custom properties для `:root`, `.dark-mode` и `.light-mode`:
 - Accent colors: `--accent-cyan: #00f2fe;`, `--accent-blue: #4facfe;`, `--accent-purple: #654ff0;`
 - Status colors: `--success: #2ed573;`, `--warning: #ffa502;`, `--danger: #ff4757;`
 - Surfaces: `--bg-primary: #0a0e17;`, `--bg-secondary: #111827;`, `--card-bg: rgba(17, 24, 39, 0.7);`
 - Blur & shadows: `--glass-blur: blur(16px);`, `--panel-shadow: 0 8px 32px 0 rgba(0, 0, 0, 0.37);`
- `product-refresh.css`: унифицирует radii, padding и sub-pixel glow effects.

5.3 Layout, навигация и Panels

- `layout.css`: трехколоночная grid-архитектура:
 - Колонка 1: `.navigation-sidebar` (260px)
 - Колонка 2: `.mail-list-pane` (320px, collapsible)
 - Колонка 3: `.mail-content-pane` (flex-grow: 1)
- `ui.css`: повторно используемые UI primitives: buttons (`.primary-btn`, `.secondary-btn`), badges, inputs, form groups.
- `modals.css`: Modal backdrop (`.modal-backdrop`), centered containers (`.modal-container`) и animations.

5.4 Стили отдельных модулей и страниц

- `dashboard.css`: tile grid (`.dashboard-grid-cards`), uplink status, quick-access cards.
- `chat.css`: chat bubbles (`.chat-bubble.sent`, `.received`), audio/file attachments, reaction picker, quote bars.
- `groupchat.css`: overrides для group chat, room header, moderation bar.
- `reader.css`: Mail Reader, header metadata, attachment cards, Composer form.
- `drive.css`: GaiaDrive tree, OPFS file list, encryption indicators, cloud status.

- `profile.css` : profile cards, mnemonic reveal grid, inactivity/privacy switches.
- `publicChannels.css` : Social Broadcast Feed, Post Cards, Comment Threads, Discovery Tiles.
- `gsn.css` : GSN Social Feed, microblogging Composer, verified Passport badges, image grid.
- `auth.css` : Login, registration и unlock cards с animated background gradients.
- `onboarding.css` : пошаговый first-user wizard.
- `security.css` : GaiaShield event logs, status indicators и operator audit tables.
- `abuseCenter.css` : Abuse cases, evidence reviewer, threshold-voting bars.
- `networkHealth.css` : Federation status tables, ping-latency displays.
- `gaiadrop.css` : формы отправки и inbox list для анонимных submission.
- `vault.css` : *Legacy*: старые стили для locked note cards.

5.5 Responsive Design и Mobile Dock

- `responsive.css` : media queries для экранов меньше 992px и 600px:
 - боковые колонки превращаются в off-canvas drawers (`.mobile-menu-open`);
 - отображается нижний mobile dock (`.mobile-navigation-dock`);
 - touch-optimized minimum heights (мин. 34px / 44px) для buttons.
- `stability-refresh.css` : defensive layout rules, scroll anchoring и защита от horizontal overflow на мобильных устройствах.

5.6 Визуальные эффекты: Nebula и Quantum Interface

- `nebula.css` : футуристические эффекты: glassmorphism, animated chrome frames (`.nebula-content-chrome`), luminous glows.
- `quantum-interface.css` : визуализация quantum-crypto state: pulsing shields, algorithm pills (`HQC-256` , `ML-KEM-1024`).

5.7 Матрица Style Files и компонентов

CSS-файл	Затрагиваемые компоненты и Views
base.css	Global DOM, App.jsx, все Panes
theme.css	Global color scheme, Dark/Light Mode switch
ui.css	AppFeedbackModals.jsx, AppModal.jsx, все buttons/forms
layout.css	NavigationSidebar.jsx, ListPane.jsx, AppMainContent.jsx
auth.css	AuthScreen.jsx, UnlockScreen.jsx, SetupWizard.jsx
landing.css	Landing page, hero banner, welcome views
listpane.css	ListPane.jsx, folder/contact/room lists
reader.css	ReaderPane.jsx, ComposerPane.jsx
chat.css	ChatPane.jsx, MessageActions.jsx
groupchat.css	GroupChatPane.jsx, GroupSettingsModal.jsx
drive.css	DrivePane.jsx (GaiaDrive)

CSS-файл	Затрагиваемые компоненты и Views
vault.css	VaultPane.jsx (Legacy)
profile.css	ProfilePane.jsx, ContactProfileModal.jsx
networkHealth.css	NetworkHealthDashboard.jsx
publicChannels.css	PublicChannelsPane.jsx, CreateChannelModal.jsx
gaiadrop.css	DropPane.jsx
modals.css	AppModalLayer.jsx, AddContactModal.jsx, CreateGroupModal.jsx
abuseCenter.css	AbuseCenter.jsx
dashboard.css	DashboardPane.jsx
security.css	SecurityCenter.jsx
gsn.css	GsnPane.jsx, все компоненты в components/chat/gsn/
onboarding.css	FirstRunOnboarding.jsx
responsive.css	MobileNavigationDock.jsx, mobile drawers во всех Panes
nebula.css	Background frames/panels в AppMainContent.jsx, DashboardPane.jsx
product-refresh.css	Refined cards/tooltips во всем Workspace
quantum-interface.css	QuantumShieldModal.jsx, security badges в Chat/Dashboard
stability-refresh.css	Global scroll containers и overflow prevention

6. Архитектура API и справочник Endpoints

Endpoint	Метод	Frontend caller	Backend handler	Service	Источник данных
/livez	GET	Browser / Monitor	operations.Monitor.Liveness	operations.Monitor	In-memory статус
/readyz	GET	Browser / Monitor	operations.Monitor.Readiness	operations.Monitor	DB Ping (database.db)
/metrics	GET	Monitoring Agent	operations.Monitor.Metrics	operations.Monitor	In-memory счетчики
/api/v1/public/version	GET	VersionBadge, NetworkHealth	Анонимный route	buildinfo	buildinfo.Current()
/api/v1/public/identity/:gaiaID	GET	api.getPublicIdentity	identity.Handler.GetPublicIdentity	identity.IdentityService	identities
/api/v1/public/trust-passport/:gaiaID	GET	api.getTrustPassport	identity.Handler.GetTrustPassport	identity.IdentityService	identities, abuse_scores
/api/v1/public/nodes	GET	api.getNodes	federation.Handler.GetNodes	federation.Service	federation_servers
/api/v1/public/node-registry/ping	POST	Внешние nodes	noderegistry.Handler.PublicPing	noderegistry.Service	node_registry_entries
/api/v1/public/node-registry/nodes	GET	NetworkHealthDashboard	noderegistry.Handler.PublicNodes	noderegistry.Service	node_registry_entries
/api/v1/public/network-health	GET	api.getNetworkHealth	networkhealth.Handler.Dashboard	networkhealth.Service	In-memory & federation_servers
/api/v1/public/gaiadrop/submit	POST	Внешний пользователь / whistleblower	gaiadrop.Handler.Submit	gaiadrop.Service	gaia_drop_submissions
/api/v1/public/smtp/ingest	POST	Внешний Mail Server	smtpbridge.Handler.Ingest	smtpbridge.Service	message_envelopes, inboxes
/api/v1/public/transparency	GET	api.getPublicTransparency	governance.Handler.GetPublicTransparency	governance.Service	transparency_snapshots
/api/v1/public/gaias-eyes/consume	POST	api.consumeGaiasEyesKey	governance.Handler.ConsumeGaiasEyesGrant	governance.Service	gaias_eyes_grants
/api/v1/public/security/health	GET	api.getPublicSecurityHealth	security.SecurityHandler.GetPublicHealth	security.SecuritySystem	security_events
/well-known/gaiacom/server	GET	Federation Peer	federation.Handler.HandleServerDiscovery	federation.Service	Конфигурация
/well-known/gaiacom/nodeinfo	GET	Federation Peer	federation.Handler.HandleNodeInfo	federation.Service	Конфигурация и build info
/well-known/gaiacom/nodes	GET	Federation Peer	noderegistry.Handler.PublicNodes	noderegistry.Service	node_registry_entries
/well-known/gaiacom/s2s/v1/forward	POST	Federation Peer	federation.Handler.HandleS2SForward	federation.Service	message_envelopes, inboxes
/_gaiacom/s2s/v1/forward	POST	Federation Peer	federation.Handler.HandleS2SForward	federation.Service	message_envelopes, inboxes
/api/v1/auth/register	POST	api.register	auth.AuthHandler.Register	auth.AuthService	users, identities

Endpoint	Метод	Frontend caller	Backend handler	Service	Источник данных
/api/v1/auth/login	POST	api.login	auth.AuthHandler.Login	auth.AuthService	users, device_sessions
/api/v1/auth/refresh	POST	api.rotateSession	auth.AuthHandler.Refresh	auth.AuthService	device_sessions
/api/v1/auth/status	GET	api.getStatus	auth.AuthHandler.GetStatus	auth.AuthService	users, identities
/api/v1/auth/change-password	POST	api.changePassword	auth.AuthHandler.ChangePassword	auth.AuthService	users
/api/v1/auth/logout	POST	api.logout	auth.AuthHandler.Logout	auth.AuthService	device_sessions
/api/v1/auth/delete-account	POST	api.deleteAccount	auth.AuthHandler.DeleteAccount	auth.AuthService	users (CASCADE)
/api/v1/auth/privacy	POST	api.updatePrivacySettings	auth.AuthHandler.UpdatePrivacy	auth.AuthService	users
/api/v1/auth/notification-preferences	GET	api.getNotificationPreferences	auth.AuthHandler.GetNotificationPreferences	auth.AuthService	user_notification_preferences
/api/v1/auth/notification-preferences	POST	api.saveNotificationPreferences	auth.AuthHandler.SaveNotificationPreferences	auth.AuthService	user_notification_preferences
/api/v1/auth/devices	GET	api.getDeviceSessions	auth.AuthHandler.ListDevices	auth.AuthService	device_sessions
/api/v1/auth/devices/revoke	POST	api.revokeDeviceSession	auth.AuthHandler.RevokeDevice	auth.AuthService	device_sessions
/api/v1/devices/pairings	POST	api.startDevicePairing	devices.Handler.Start	devices.Service	device_pairings
/api/v1/devices/pairings/:id	GET	api.getDevicePairing	devices.Handler.Get	devices.Service	device_pairings
/api/v1/devices/pairings/:id/approve	POST	api.approveDevicePairing	devices.Handler.Approve	devices.Service	device_pairings, device_keys
/api/v1/devices/pairings/:id/consume	POST	api.consumeDevicePairing	devices.Handler.Consume	devices.Service	device_pairings
/api/v1/devices/keys	GET	api.getDeviceKeys	devices.Handler.ListKeys	devices.Service	device_keys
/api/v1/devices/recipient-keys	GET	api.getRecipientDeviceKeys	devices.Handler.RecipientKeys	devices.Service	device_keys
/api/v1/devices/keys/:id/revoke	POST	api.revokeDeviceKey	devices.Handler.RevokeKey	devices.Service	device_keys
/api/v1/identity/create	POST	api.createIdentity	identity.Handler.CreateIdentity	identity.IdentityService	identities
/api/v1/identity/me	GET	api.getMyIdentities	identity.Handler.GetMyIdentities	identity.IdentityService	identities
/api/v1/identity/human-proof	POST	api.saveIdentityHumanProof	identity.Handler.SaveHumanProof	identity.IdentityService	identities
/api/v1/messaging/send	POST	api.sendMessage	messaging.Handler.SendMessage	messaging.MessagingService	message_envelopes, inboxes
/api/v1/smtp/send	POST	api.sendSmtpMail	smtpbridge.Handler.Send	smtpbridge.Service	Outgoing SMTP / message_envelopes
/api/v1/messaging/inbox	GET	api.getInbox	messaging.Handler.GetInbox	messaging.MessagingService	inboxes, message_envelopes

Endpoint	Метод	Frontend caller	Backend handler	Service	Источник данных
/api/v1/messaging/read	POST	api.markMessagesRead	messaging.Handler.MarkRead	messaging.MessagingService	inboxes, message_read_receipts
/api/v1/messaging/edit	POST	api.editDirectMessage	messaging.Handler.EditMessage	messaging.MessagingService	message_envelopes
/api/v1/messaging/reaction	POST	api.toggleMessageReaction	messaging.Handler.ToggleReaction	messaging.MessagingService	message_reactions
/api/v1/messaging/proof	GET	api.getMessageProof	messaging.Handler.GetMessageProof	messaging.MessagingService	message_proofs
/api/v1/messaging/delete	POST	api.deleteInboxMessage	messaging.Handler.DeleteInboxMessage	messaging.MessagingService	inboxes, message_envelopes
/api/v1/messaging/clear	POST	api.clearInboxConversation	messaging.Handler.ClearInboxConversation	messaging.MessagingService	inboxes
/api/v1/presence/heartbeat	POST	api.sendPresenceHeartbeat	presence.Handler.Heartbeat	presence.Service	identity_presence
/api/v1/presence/status	GET	api.getPresenceStatus	presence.Handler.Status	presence.Service	identity_presence
/api/v1/presence/typing	POST	api.updateTypingStatus	presence.Handler.UpdateTyping	presence.Service	In-memory typing map
/api/v1/presence/typing	GET	api.getTypingStatus	presence.Handler.TypingStatus	presence.Service	In-memory typing map
/api/v1/mailbox/messages	GET	api.getMailboxMessages	mailbox.Handler.ListMessages	mailbox.Service	mailbox_states, message_envelopes
/api/v1/mailbox/state	POST	api.updateMailboxStates	mailbox.Handler.UpdateStates	mailbox.Service	mailbox_states
/api/v1/mailbox/drafts	GET	api.getMailDrafts	mailbox.Handler.ListDrafts	mailbox.Service	mail_drafts
/api/v1/mailbox/drafts/save	POST	api.saveMailDraft	mailbox.Handler.SaveDraft	mailbox.Service	mail_drafts
/api/v1/mailbox/drafts/delete	POST	api.deleteMailDraft	mailbox.Handler.DeleteDraft	mailbox.Service	mail_drafts
/api/v1/mailbox/labels	GET	api.getMailLabels	mailbox.Handler.ListLabels	mailbox.Service	mail_labels
/api/v1/mailbox/labels/save	POST	api.saveMailLabel	mailbox.Handler.SaveLabel	mailbox.Service	mail_labels
/api/v1/mailbox/contacts	GET	api.getMailContacts	mailbox.Handler.ListContacts	mailbox.Service	mail_contacts
/api/v1/mailbox/contacts/save	POST	api.saveMailContact	mailbox.Handler.SaveContact	mailbox.Service	mail_contacts
/api/v1/mailbox/filters	GET	api.getMailFilters	mailbox.Handler.ListFilters	mailbox.Service	mail_filter_rules
/api/v1/mailbox/filters/save	POST	api.saveMailFilter	mailbox.Handler.SaveFilter	mailbox.Service	mail_filter_rules
/api/v1/mailbox/settings	GET	api.getMailSettings	mailbox.Handler.GetSettings	mailbox.Service	mail_settings
/api/v1/mailbox/settings	POST	api.saveMailSettings	mailbox.Handler.SaveSettings	mailbox.Service	mail_settings

Endpoint	Метод	Frontend caller	Backend handler	Service	Источник данных
/api/v1/search/global	GET	api.globalSearch	mailbox.Handler.GlobalSearch	mailbox.Service	message_envelopes, mail_contacts
/api/v1/storage/init	POST	api.initUpload	storage.Handler.InitUpload	storage.Service	file_metadata
/api/v1/storage/chunk	POST	api.uploadChunk	storage.Handler.UploadChunk	storage.Service	file_chunks, ObjectStore
/api/v1/storage/complete	POST	api.completeUpload	storage.Handler.CompleteUpload	storage.Service	file_metadata
/api/v1/storage/grant	POST	api.grantFileAccess	storage.Handler.GrantAccess	storage.Service	file_access_grants
/api/v1/storage/download/:fileId	GET	api.downloadFileAttachment	storage.Handler.DownloadFile	storage.Service	file_metadata, ObjectStore
/api/v1/reports/submit	POST	api.submitReport	trustmesh.Handler.SubmitReport	trustmesh.Service	reports, abuse_scores
/api/v1/gaiadrop/inbox	GET	api.getGaiaDropInbox	gaiadrop.Handler.ListInbox	gaiadrop.Service	gaia_drop_submissions
/api/v1/gaiadrop/read	POST	api.markGaiaDropRead	gaiadrop.Handler.MarkRead	gaiadrop.Service	gaia_drop_submissions
/api/v1/gaiadrop/delete	POST	api.deleteGaiaDrop	gaiadrop.Handler.Delete	gaiadrop.Service	gaia_drop_submissions
/api/v1/gsn/posts	POST	api.createGsnPost	gsn.Handler.CreatePost	gsn.Service	gsn_posts
/api/v1/gsn/posts/:id	DELETE	api.deleteGsnPost	gsn.Handler.DeletePost	gsn.Service	gsn_posts
/api/v1/gsn/feed/node	GET	api.getGsnFeedNode	gsn.Handler.GetFeedNode	gsn.Service	gsn_posts
/api/v1/gsn/feed/following	GET	api.getGsnFeedFollowing	gsn.Handler.GetFeedFollowing	gsn.Service	gsn_posts, gsn_follows
/api/v1/gsn/posts/:id/react	POST	api.reactToGsnPost	gsn.Handler.ReactToPost	gsn.Service	gsn_post_reactions
/api/v1/gsn/posts/:id/comment	POST	api.addGsnComment	gsn.Handler.AddComment	gsn.Service	gsn_post_comments
/api/v1/gsn/posts/:id/comments	GET	api.getGsnComments	gsn.Handler.GetComments	gsn.Service	gsn_post_comments
/api/v1/gsn/posts/:id/comments/:cId	DELETE	api.deleteGsnComment	gsn.Handler.DeleteComment	gsn.Service	gsn_post_comments
/api/v1/gsn/follow	POST	api.followGsnUser	gsn.Handler.FollowUser	gsn.Service	gsn_follows
/api/v1/gsn/unfollow	POST	api.unfollowGsnUser	gsn.Handler.UnfollowUser	gsn.Service	gsn_follows
/api/v1/gsn/profile/:gaia_id	GET	api.getGsnProfile	gsn.Handler.GetProfile	gsn.Service	gsn_profiles
/api/v1/gsn/profile	POST	api.updateGsnProfile	gsn.Handler.UpdateProfile	gsn.Service	gsn_profiles
/api/v1/rooms/create	POST	api.createRoom	room.Handler.CreateRoom	room.Service	rooms, room_members, channels
/api/v1/rooms/update	POST	api.updateRoom	room.Handler.UpdateRoom	room.Service	rooms
/api/v1/rooms	GET	api.getRooms	room.Handler.GetRooms	room.Service	rooms, room_members

Endpoint	Метод	Frontend caller	Backend handler	Service	Источник данных
/api/v1/rooms/join	POST	api.joinRoom	room.Handler.JoinRoomByHash	room.Service	room_members
/api/v1/rooms/leave	POST	api.leaveRoom	room.Handler.LeaveRoom	room.Service	room_members
/api/v1/rooms/channels	POST	api.createChannel	room.Handler.CreateChannel	room.Service	channels
/api/v1/rooms/channels	GET	api.getChannels	room.Handler.GetChannels	room.Service	channels
/api/v1/rooms/members/role	POST	api.updateMemberRole	room.Handler.UpdateMemberRole	room.Service	room_members
/api/v1/rooms/delete	POST	api.deleteRoom	room.Handler.DeleteRoom	room.Service	rooms (CASCADE)
/api/v1/rooms/search	GET	api.searchPublicRooms	room.Handler.SearchPublicRooms	room.Service	rooms
/api/v1/rooms/members/kick	POST	api.kickRoomMember	room.Handler.KickMember	room.Service	room_members
/api/v1/rooms/transfer-ownership	POST	api.transferRoomOwnership	room.Handler.TransferOwnership	room.Service	rooms, room_members
/api/v1/rooms/pins	GET	api.getRoomPinnedMessages	room.Handler.GetRoomPinnedMessages	room.Service	room_pinned_messages
/api/v1/rooms/pins/toggle	POST	api.toggleRoomMessagePin	room.Handler.ToggleRoomMessagePin	room.Service	room_pinned_messages
/api/v1/rooms/invites/create	POST	api.createRoomInviteLink	room.Handler.CreateRoomInviteLink	room.Service	room_invite_links
/api/v1/rooms/invites/join	POST	api.joinRoomViaInviteLink	room.Handler.JoinRoomViaInviteLink	room.Service	room_invite_links, room_members
/api/v1/rooms/join-requests	GET	api.getRoomJoinRequests	room.Handler.GetRoomJoinRequests	room.Service	room_join_requests
/api/v1/rooms/join-requests/create	POST	api.createRoomJoinRequest	room.Handler.CreateRoomJoinRequest	room.Service	room_join_requests
/api/v1/rooms/join-requests/moderate	POST	api.moderateRoomJoinRequest	room.Handler.ModerateRoomJoinRequest	room.Service	room_join_requests, room_members
/api/v1/rooms/moderation-logs	GET	api.getRoomModerationLogs	room.Handler.GetRoomModerationLogs	room.Service	room_moderation_logs
/api/v1/public-channels	GET	api.getPublicChannels	publicchannels.Handler.List	publicchannels.Service	public_channels
/api/v1/public-channels/create	POST	api.createPublicChannel	publicchannels.Handler.Create	publicchannels.Service	public_channels, admins
/api/v1/public-channels/update	POST	api.updatePublicChannel	publicchannels.Handler.Update	publicchannels.Service	public_channels
/api/v1/public-channels/comments	POST	api.updatePublicChannelComments	publicchannels.Handler.UpdateComments	publicchannels.Service	public_channels
/api/v1/public-channels/delete	POST	api.deletePublicChannel	publicchannels.Handler.Delete	publicchannels.Service	public_channels
/api/v1/public-channels/subscribe	POST	api.subscribePublicChannel	publicchannels.Handler.Subscribe	publicchannels.Service	public_channel_subscribers
/api/v1/public-channels/unsubscribe	POST	api.unsubscribePublicChannel	publicchannels.Handler.Unsubscribe	publicchannels.Service	public_channel_subscribers

Endpoint	Метод	Frontend caller	Backend handler	Service	Источник данных
/api/v1/public-channels/posts	GET	api.getPublicChannelPosts	publicchannels.Handler.ListPosts	publicchannels.Service	public_channel_posts
/api/v1/public-channels/posts/create	POST	api.createPublicChannelPost	publicchannels.Handler.CreatePost	publicchannels.Service	public_channel_posts
/api/v1/public-channels/posts/reaction	POST	api.togglePublicChannelPostReaction	publicchannels.Handler.TogglePostReaction	publicchannels.Service	public_channel_post_reactions
/api/v1/public-channels/posts/comment	POST	api.createPublicChannelPostComment	publicchannels.Handler.CreatePostComment	publicchannels.Service	public_channel_post_comments
/api/v1/public-channels/posts/pin	POST	api.updatePublicChannelPostPin	publicchannels.Handler.UpdatePostPin	publicchannels.Service	public_channel_posts
/api/v1/public-channels/block	POST	api.blockPublicChannel	publicchannels.Handler.Block	publicchannels.Service	public_channel_blocks
/api/v1/public-channels/unblock	POST	api.unblockPublicChannel	publicchannels.Handler.Unblock	publicchannels.Service	public_channel_blocks
/api/v1/public-channels/discover	GET	api.discoverPublicChannels	publicchannels.Handler.Discover	publicchannels.Service	public_channels
/api/v1/public-channels/posts/comments/delete	POST	api.deleteChannelComment	publicchannels.Handler.DeleteComment	publicchannels.Service	public_channel_post_comments
/api/v1/public-channels/posts/comments/moderate	POST	api.moderateChannelComment	publicchannels.Handler.ModerateComment	publicchannels.Service	public_channel_post_comments
/api/v1/governance/roles	GET	api.getGovernanceRoles	governance.Handler.CheckRoles	governance.Service	role_credentials
/api/v1/reports	POST	api.submitAbuseReport	governance.Handler.CreateReport	governance.Service	abuse_cases, abuse_case_events
/api/v1/reports/mine	GET	api.getMyReports	governance.Handler.GetMyReports	governance.Service	abuse_cases
/api/v1/reports/:caseID	GET	api.getReportDetail	governance.Handler.GetReportDetail	governance.Service	abuse_cases, events, packages
/api/v1/reports/:caseID/disclosures	POST	api.createCaseDisclosure	governance.Handler.CreateCaseDisclosure	governance.Service	abuse_disclosure_packages
/api/v1/reviewer/cases/:caseID/disclosure-requests	POST	api.requestCaseDisclosure	governance.Handler.RequestCaseDisclosure	governance.Service	abuse_disclosure_requests
/api/v1/reports/:caseID/appeal	POST	api.submitAppeal	governance.Handler.AppealReport	governance.Service	abuse_appeals
/api/v1/gaias-eyes/grants	POST	api.createGaiasEyesGrant	governance.Handler.CreateGaiasEyesGrant	governance.Service	gaias_eyes_grants
/api/v1/reviewer/cases	GET	api.getReviewerQueue	governance.Handler.GetReviewerQueue	governance.Service	abuse_cases
/api/v1/reviewer/cases/:caseID/review	POST	api.submitReview	governance.Handler.SubmitReview	governance.Service	abuse_reviews
/api/v1/node/abuse/queue	GET	api.getNodeOperatorQueue	governance.Handler.GetNodeOperatorQueue	governance.Service	abuse_cases
/api/v1/node/abuse/actions	POST	api.applyNodeOperatorAction	governance.Handler.ApplyNodeOperatorAction	governance.Service	abuse_actions
/api/v1/node/transparency/snapshot	POST	api.createTransparencySnapshot	governance.Handler.CreateTransparencySnapshot	governance.Service	transparency_snapshots

Endpoint	Метод	Frontend caller	Backend handler	Service	Источник данных
/api/v1/security/me/summary	GET	api.getSecuritySummary	security.SecurityHandler.GetMySummary	security.SecuritySystem	security_events
/api/v1/security/me/events	GET	api.getSecurityEvents	security.SecurityHandler.GetMyEvents	security.SecuritySystem	security_events
/api/v1/security/me/events/:id/ack	POST	api.acknowledgeSecurityEvent	security.SecurityHandler.AcknowledgeEvent	security.SecuritySystem	security_user_acknowledgements
/api/v1/security/me/report	GET	api.exportSecurityReport	security.SecurityHandler.ExportReport	security.SecuritySystem	security_events, audit_chain
/api/v1/node/security/summary	GET	api.getNodeSecuritySummary	security.SecurityHandler.GetNodeSummary	security.SecuritySystem	security_events, rules
/api/v1/node/security/events	GET	api.getNodeSecurityEvents	security.SecurityHandler.GetNodeEvents	security.SecuritySystem	security_events
/api/v1/node/registry/summary	GET	api.getNodeRegistrySummary	noderegistry.Handler.GetSummary	noderegistry.Service	node_registry_entries
/api/v1/node/registry/secrets	POST	api.generateNodeRegistrySecrets	noderegistry.Handler.GenerateSecrets	noderegistry.Service	In-memory secret generator
/api/v1/node/registry/ping-main	POST	api.pingNodeRegistryMain	noderegistry.Handler.PingMain	noderegistry.Service	Federation ping к main node
/api/v1/node/registry/:domain/status	POST	api.updateNodeRegistryStatus	noderegistry.Handler.UpdateStatus	noderegistry.Service	node_registry_entries

7. Системные потоки данных

Поток данных 1: создание Identity и генерация Dual-PQC ключей

```
Пользователь открывает SetupWizard.jsx / AuthScreen.jsx
↓
Вводит master password и желаемый GaiaID (@alice:node.domain)
↓
Frontend: crypto.js генерирует 12-словную BIP-39 mnemonic
↓
BIP-39 mnemonic
├─> PBKDF2 -> X25519 identity key pair
├─> PBKDF2 -> Ed25519 signing key pair
├─> Sovereign Worker -> ML-KEM-1024 KEM key pair
└─> Sovereign Worker / Tauri Host -> HQC-256 KEM key pair
↓
Private keys шифруются client-side с master password (PBKDF2/AES-GCM)
и сохраняются в local browser storage / Desktop Vault
↓
POST /api/v1/identity/create
  Payload: { gaiaId, displayName, publicRecord: { x25519, ed25519, ml_kem_1024, hqc_256 } }
↓
Backend: identity.IdentityHandler -> identity.IdentityService
↓
Integrity checks и GaiaID validation согласно RFC specification
↓
Repository: запись в identities (is_active = 1)
↓
200 OK (Identity зарегистрирована, public keys опубликованы)
```

Поток данных 2: отправка и получение E2EE Quantum Chat сообщения

```
Alice вводит сообщение для Bob (@bob:remote.domain)
↓
ChatPane.jsx вызывает useChat.js -> sendMessage()
↓
Client получает public keys Bob: GET /api/v1/public/identity/@bob:remote.domain
↓
Сверка с локальным Key History Store (Key Change Detection)
↓
Sovereign Crypto Engine (Dual-PQC KEM):
  1. X25519 Diffie-Hellman shared secret (ss_classic)
  2. ML-KEM-1024 encapsulation -> kem_ciphertext + ss_mlkem
  3. HQC-256 encapsulation -> hqc_ciphertext + ss_hqc
↓
Combiner: root_secret = HKDF-SHA3-512(ss_classic || ss_mlkem || ss_hqc, salt, transcript_aad)
↓
Cascade encryption:
  Twofish-256-EAX затем AES-256-GCM (Accelerated)
  либо четырехступенчатый cascade с Serpent и XChaCha20 (Top Secret)
↓
Envelope подписывается private Ed25519 / ML-DSA-87 key Alice
↓
POST /api/v1/messaging/send (Server видит ТОЛЬКО ciphertext, KEM ciphertexts, nonces, hashes)
↓
Home Server Alice: сохраняет в message_envelopes и inboxes
↓
Проверка: Bob локальный или находится на remote node?
├─> Local: запись в Inbox index Bob
└─> Remote: federation queue создает signed PDU -> S2S POST на Server Bob
↓
Client Bob запрашивает GET /api/v1/messaging/inbox
↓
Sovereign Worker Bob:
  1. Decapsulation ML-KEM-1024 ciphertext с private ML-KEM key Bob
  2. Decapsulation HQC-256 ciphertext с private HQC key Bob
```



```
3. X25519 DH с ephemeral key
4. Root Secret derivation и cascade decryption
↓
Plaintext message отображается в ChatPane.jsx Bob
↓
Автоматический запрос: POST /api/v1/messaging/read (read receipt)
```

Поток данных 3: локальное хранение GaiaDrive и Cloud Sync

```
Пользователь переносит файл в DrivePane.jsx
↓
useDrive.jsx перехватывает файл
↓
Client-side encryption:
- генерируется случайный 256-bit AES-GCM key & IV
- file payload шифруется в browser storage
- raw ciphertext сохраняется в Origin Private File System (OPFS)
- encrypted metadata index обновляется в localStorage
↓
Оptionальный Cloud Sync (≤ 20 MB, TTL 14 дней):
↓
POST /api/v1/storage/init (fileName, fileSize, mimeType, fileHash)
↓
Backend генерирует file_id и проверяет storage quota пользователя
↓
Client разбивает encrypted payload на chunks по 1 MB и отправляет:
POST /api/v1/storage/chunk (Multipart FormData с chunkHash & chunkIndex)
↓
StorageService проверяет SHA-256 hash chunk
├→ Local: запись в secured storage jail (uploads/<fileId>/chunk_<index>)
└→ S3: запись напрямую в S3/MinIO bucket
↓
POST /api/v1/storage/complete
↓
StorageService собирает metadata и устанавливает status = 'completed'
```

Поток данных 4: S2S Federation с Signed PDUs и SSRF-защитой

```
Node A хочет передать сообщение Node B
↓
FederationService создает PDU:
{ pdu_id, origin: "node-a.org", destination: "node-b.org", type: "message", payload: envelope }
↓
PDU подписывается Ed25519 server private key Node A
↓
HTTP POST к Node B: /.well-known/gaiacom/s2s/v1/forward
Header: Authorization: X-Gaia-S2S-V1 Signature="...",KeyId="node-a.org",Timestamp="..."
↓
Node B EdgeShield & FederationHandler:
1. Проверка временного окна (|timestamp - now| < 300s)
2. SSRF firewall: resolve node-a.org IP; private/loopback IP блокируются
3. Replay guard: проверяет federation_replay_guard; duplicate pdu_id немедленно отклоняется
4. Signature check: проверка подписи по Ed25519 public key node-a.org
↓
200 OK PDU Accepted
↓
Node B помещает сообщение в локальный Inbox получателя
```

Поток данных 5: Abuse Report, Gaia's Eyes Grant и Reviewer Queue

```
Получатель замечает harassment или spam в ReaderPane.jsx / ChatPane.jsx
↓
Нажатие "Report" открывает Abuse Center dialog
↓
Client генерирует GaiaProof (ciphertext hash, sender signature, server receipt timestamp)
↓
POST /api/v1/reports
  Payload: { targetGaiaID, category, severity, messageId, gaiaProof }
↓
Backend: governance.Handler -> governance.Service
↓
Создается abuse_case (status: 'pending_review')
↓
Reviewer открывает AbuseCenter.jsx -> GET /api/v1/reviewer/cases
↓
Reviewer запрашивает Evidence: POST /reviewer/cases/:id/disclosure-requests
↓
Reported user или reporter предоставляет временный доступ через Gaia's Eyes:
POST /api/v1/gaias-eyes/grants
  Генерируется time-limited one-time token (TTL: 15 минут)
↓
Reviewer использует token через POST /api/v1/public/gaias-eyes/consume
↓
Reviewer голосует: POST /reviewer/cases/:id/review (categoryVote, severityVote, recommendation)
↓
Threshold достигнут (например 2 из 3 Reviewers):
Node Operator применяет Action (friction throttling или temporary quarantine)
```

Поток данных 6: Security Event GaiaShield и Audit Chain

```
Unauthorized API access, BOLA attempt или rate-limit violation
↓
Backend: EdgeShieldMiddleware или SecuritySystem фиксирует нарушение
↓
SecuritySystem.RecordEvent():
  1. Classification: severity (INFO, WARNING, CRITICAL), category (AUTH, API, FEDERATION)
  2. Privacy redaction: IP address и User-Agent хешируются, tokens удаляются
  3. Audit chaining:
      event_hash = SHA256(previous_hash || event_id || timestamp || category || severity)
↓
Atomic transaction в SQLite:
  - INSERT INTO security_events
  - INSERT INTO security_audit_chain (event_id, previous_hash, event_hash)
↓
SQLite triggers обеспечивают immutability:
  trg_security_events_immutable_update блокирует изменения
  trg_security_events_no_delete блокирует удаления
↓
Live warning пользователю через /api/v1/security/me/events
либо Node Operator через SecurityCenter.jsx
```

8. Общие и Core-компоненты

Следующие модули и файлы являются фундаментальными элементами всей системы и повторно используются многими модулями.

1. Frontend API Client (`Frontend/frontend/src/api.js`)

CORE / SHARED DEPENDENCY

- **Назначение:** инкапсулирует REST calls, управляет authentication headers (`Bearer JWT`), автоматической token rotation (`/api/v1/auth/refresh`), desktop detection (`isDesktopRuntime`) и единообразной обработкой ошибок.
- **Используется:** почти всеми frontend panes, hooks (`useChat` , `useEmails` , `useDrive` , `usePublicChannels` , `useGsn`) и modals.

2. Frontend Crypto Orchestrator (`Frontend/frontend/src/crypto.js` и `sovereignCrypto.js`)

CORE / SHARED DEPENDENCY

- **Назначение:** центральная точка для криптографических операций Client. Инкапсулирует Noble curves, BIP-39 mnemonic recovery, Web Worker orchestration и Tauri IPC commands.
- **Используется:** `AuthScreen.jsx` , `ChatPane.jsx` , `DrivePane.jsx` , `ComposerPane.jsx` , `GsnPane.jsx` , `useGaiaAuth.js` .

3. Shared Rust Multi-Cipher Core (`SovereignCryptoCore/src/lib.rs`)

CORE / SHARED DEPENDENCY

- **Назначение:** platform-independent, memory-safe реализация authenticated cipher cascades.
- **Используется:** `DesktopClient/src-auri` (native compile) и `SovereignCryptoWasm` (WebAssembly для Browser Worker).

4. Zero-Dependency Core Primitives (`Core/rust/gaiacore/src/lib.rs`)

CORE / SHARED DEPENDENCY

- **Назначение:** минимальная Rust library только на standard library для invariants: GaiaID validation, constant-time byte comparison, Envelope parsing и TrustMesh formulas.
- **Используется:** Rust cores и Android kernel через JNI.

5. Backend HTTP Router & Middleware Engine (`Backend/httpx/`)

CORE / SHARED DEPENDENCY

- **Назначение:** легковесный HTTP framework без Gin на базе Go `net/http` ; CORS control, strict security headers и JSON serialization.
- **Используется:** `Backend/routes.go` , всеми domain handlers и operations monitoring.

6. Backend Persistence Adapter (Backend/repository/sql_store.go)

CORE / SHARED DEPENDENCY

- **Назначение:** central wrapper вокруг `*sql.DB`; определяет transaction boundaries, isolation levels и соединяет 25 domain-specific repository частей.
- **Используется:** всеми Backend Services для relational persistence.

7. Global Domain Model (Backend/models/models.go)

CORE / SHARED DEPENDENCY

- **Назначение:** все общесистемные data structures, Envelope types, DTOs и database-row definitions.
- **Используется:** всеми Go packages Backend (`auth`, `messaging`, `federation`, `room`, `mailbox` и т. д.).

8. GaiaShield Security Subsystem (Backend/internal/security/shield.go)

CORE / SHARED DEPENDENCY

- **Назначение:** центральная система auditing, redaction чувствительных данных, IDS rule monitoring и rate limiting.
- **Используется:** `routes.go`, `authHandler`, `messagingHandler`, `storageHandler`, `fedHandler`.

9. Secure Serialization & Type Utilities (Frontend/frontend/src/utls/safeJson.js , uuid.js , payload.js)

CORE / SHARED DEPENDENCY

- **Назначение:** защита от prototype pollution при parsing untrusted payloads, строгая RFC-4122 UUID validation и sanitized data structures.
- **Используется:** почти всеми frontend components и hooks.

10. Internationalization Engine (Frontend/frontend/src/utls/i18n.jsx)

CORE / SHARED DEPENDENCY

- **Назначение:** language contexts, translation functions (`t()`) и переключение 10+ языков.
- **Используется:** всеми пользовательскими интерфейсами и dialogs.

9. Архитектурные связи и Mermaid-диаграммы

Диаграмма 1: общая архитектура системы

```
graph TB
    subgraph Client_Layer ["Client Layer (конечные устройства)"]
        subgraph Web_Client ["Web Client (Vite / React 18)"]
            ReactUI["React UI (Panels & Modals)"]
            WebCryptoWorker["Web Worker (liboqs HQC-256)"]
            SovereignWASM["SovereignCryptoWasm (Rust Cascade)"]
            OPFS["OPFS (локальный GaiaDrive Storage)"]
        end
        end
        subgraph Desktop_Client ["Desktop Client (Tauri 2 Shell)"]
            TauriHost["Tauri 2 Rust Host"]
            NativeHQC["Native pqcrypto-hqc"]
            NativeCascade["Native SovereignCryptoCore"]
        end
        end
        subgraph Mobile_Client ["Mobile Client (Android)"]
            ComposeUI["Jetpack Compose UI"]
            AndroidKernel["Platform Kernel"]
            AndroidKeystore["Keystore / StrongBox"]
            GoMobileNode["Embedded Go Node (Backend/mobileapi)"]
        end
        end
    end

    subgraph Network_Edge ["Network Edge & Ingress"]
        NGINX["NGINX Reverse Proxy (TLS 1.3, CSP, Frame Deny)"]
    end

    subgraph Backend_Application ["Go Backend Application (Port :8080)"]
        HTTPX["httpx Engine (Router, CORS, Security Headers)"]
        GaiaShield["GaiaShield (Audit Chain & Privacy Redactor)"]
        subgraph Core_Services ["Core Services"]
            AuthSvc["Auth & Device Service"]
            MsgSvc["Messaging Service"]
            MailSvc["Mailbox & Scheduler Service"]
            RoomSvc["Room & Channel Service"]
            StorageSvc["Storage Service (Quotas & Chunks)"]
            FedSvc["Federation S2S Service"]
            GovSvc["Governance & Abuse Service"]
            GsnSvc["GSN Social Service"]
        end
    end

    subgraph Persistence_Layer ["Persistence & Storage Layer"]
        SQLite["SQLite Engine (WAL Mode, Max 4/32 Conns)"]
        LocalFS["Local Chunk Path (uploads/)"]
        S3Store["S3 / MinIO Object Store (опционально)"]
    end

    subgraph External_Network ["Federated Network"]
        RemoteNode["Remote GaiaCom Node"]
        SMTPGateway["Classic SMTP Email Server"]
    end

    ReactUI --> WebCryptoWorker
    WebCryptoWorker --> SovereignWASM
    ReactUI --> OPFS
    ReactUI -->|HTTPS / API| NGINX
    TauriHost --> NativeHQC
    TauriHost --> NativeCascade
    TauriHost -->|IPC Bridge| ReactUI
    ComposeUI --> AndroidKernel
    AndroidKernel --> AndroidKeystore
    AndroidKernel --> GoMobileNode
    NGINX -->|Reverse Proxy :8080| HTTPX
    HTTPX --> GaiaShield
    GaiaShield --> Core_Services
    AuthSvc --> SQLite
```

```

MsgSvc --> SQLite
MailSvc --> SQLite
RoomSvc --> SQLite
GovSvc --> SQLite
GsnSvc --> SQLite
StorageSvc --> SQLite
StorageSvc --> LocalFS
StorageSvc --> S3Store
FedSvc <-->|Signed PDUs / S2S| RemoteNode
Core_Services -->|SMTP Ingest / Egress| SMTPGateway

```

Диаграмма 2: архитектура Dashboard и Navigation

```

graph LR
    User["Действия пользователя"] --> NavSidebar["NavigationSidebar.jsx"]
    subgraph Navigation_Sections ["Категории Navigation"]
        NavSidebar --> WorkspaceSec["Workspace"]
        NavSidebar --> NetworkSec["Network"]
        NavSidebar --> PersonalSec["Personal"]
    end
    end
    subgraph Workspace_Views ["Workspace Views"]
        WorkspaceSec --> DashboardView["DashboardPane.jsx (Command Center)"]
        WorkspaceSec --> GaiaMailView["GaiaMail (Inbox, Sent, Drafts и т. д.)"]
        WorkspaceSec --> SmtMailView["SMTP Mail (Legacy Ingest / Outbox)"]
        WorkspaceSec --> ChatView["ChatPane.jsx (Quantum Chat 1-to-1)"]
        WorkspaceSec --> GroupsView["GroupChatPane.jsx (Rooms & Channels)"]
    end
    end
    subgraph Network_Views ["Network Views"]
        NetworkSec --> ChannelsView["PublicChannelsPane.jsx (Broadcasts)"]
        NetworkSec --> GsnView["GsnPane.jsx (GaiaSocialNetwork)"]
        NetworkSec --> NetHealthView["NetworkHealthDashboard.jsx"]
        NetworkSec --> SecCenterView["SecurityCenter.jsx (GaiaShield)"]
        NetworkSec --> AbuseCenterView["AbuseCenter.jsx (Abuse Center)"]
    end
    end
    subgraph Personal_Views ["Personal Views"]
        PersonalSec --> ContactsView["ListPane.jsx (Address Book)"]
        PersonalSec --> ProfileView["ProfilePane.jsx (Keys, Mnemonic, Privacy)"]
        PersonalSec --> DriveView["DrivePane.jsx (GaiaDrive / Vault)"]
        PersonalSec --> DropView["DropPane.jsx (GaiaDrop Inbox)"]
    end
    end
    subgraph Orchestration ["Renderer & Modals"]
        AppMainContent["AppMainContent.jsx (Dynamic View Switcher)"]
        AppModalLayer["AppModalLayer.jsx (Modals & Overlays)"]
    end
    end
    Workspace_Views --> AppMainContent
    Network_Views --> AppMainContent
    Personal_Views --> AppMainContent
    AppMainContent --> AppModalLayer

```

Диаграмма 3: граф зависимостей модулей

```

graph TD
    SovereignCrypto["Модуль 1: Sovereign Cryptographic Engine"]
    AuthModule["Модуль 2: Authentication & Vault"]
    IdentityModule["Модуль 3: Identity & Trust Passport"]
    MessagingModule["Модуль 4: Quantum Chat (Direct E2EE)"]
    RoomModule["Модуль 5: Group Chats (Rooms & Channels)"]
    MailboxModule["Модуль 6: GaiaMail & Mailbox States"]
    SmtModule["Модуль 7: SMTP Legacy Bridge"]
    ChannelsModule["Модуль 8: Public Channels"]
    GsnModule["Модуль 9: GSN Social Network"]
    DriveModule["Модуль 10: GaiaDrive & Chunk Storage"]
    DropModule["Модуль 11: GaiaDrop Submissions"]
    FedModule["Модуль 12: Federation S2S Engine"]
    RegistryModule["Модуль 13: Node Registry"]
    TrustMeshModule["Модуль 14: TrustMesh Abuse Scoring"]

```

```

GovModule["Модуль 15: Governance & Gaia's Eyes"]
ShieldModule["Модуль 16: GaiaShield Security System"]
AuthModule --> SovereignCrypto
IdentityModule --> AuthModule
IdentityModule --> SovereignCrypto
MessagingModule --> SovereignCrypto
MessagingModule --> IdentityModule
MessagingModule --> ShieldModule
RoomModule --> MessagingModule
RoomModule --> IdentityModule
MailboxModule --> MessagingModule
MailboxModule --> IdentityModule
SmtpModule --> MailboxModule
SmtpModule --> ShieldModule
ChannelsModule --> IdentityModule
GsnModule --> IdentityModule
GsnModule --> SovereignCrypto
DriveModule --> SovereignCrypto
DriveModule --> AuthModule
DropModule --> IdentityModule
DropModule --> ShieldModule
FedModule --> MessagingModule
FedModule --> ShieldModule
RegistryModule --> FedModule
TrustMeshModule --> MessagingModule
GovModule --> TrustMeshModule
GovModule --> ShieldModule

```

Диаграмма 4: Backend / API / Database Pipeline

```

sequenceDiagram
    autonumber
    actor Client as Frontend Client (React/Tauri)
    participant Nginx as NGINX Reverse Proxy
    participant HTTPX as httpx Router & Middleware
    participant Shield as GaiaShield (Edge Guard)
    participant Handler as Domain Handler (например Messaging)
    participant Service as Domain Service (например MessagingService)
    participant Repo as SQLStore (Repository Adapter)
    participant DB as SQLite Database Engine

    Client->>Nginx: HTTPS POST /api/v1/messaging/send (Bearer Token, Ciphertext)
    Nginx->>HTTPX: Proxy Pass 127.0.0.1:8080
    HTTPX->>Shield: SecurityHeaders & EdgeShieldMiddleware
    Shield->>Handler: Передача authenticated handler
    Handler->>Service: SendMessage(senderIdentityId, recipientIds, envelope)
    Service->>Repo: CreateEnvelope(envelope) & DeliverToInbox(inbox)
    Repo->>DB: BEGIN IMMEDIATE TRANSACTION
    Repo->>DB: INSERT INTO message_envelopes (...)
    Repo->>DB: INSERT INTO inboxes (...)
    Repo->>DB: INSERT INTO message_proofs (...)
    DB-->>Repo: Commit Transaction Success
    Repo-->>Service: Persistence подтвержден
    Service-->>Handler: Message успешно routed
    Handler-->>HTTPX: httpx.WriteJSON(w, 200, {status: 'delivered'})
    HTTPX-->>Nginx: Response с security headers
    Nginx-->>Client: 200 OK

```

10. Ссылки на файлы

Для технической прослеживаемости конкретные детали реализации перечислены ниже:

- `Backend/repository/sql_store.go`: определяет `SQLStore` как concrete adapter вокруг `*sql.DB` и реализует интерфейс `repository.Store` из `Backend/repository/repository.go`.
- `Backend/database/database.go`: конфигурирует SQLite WAL через `PRAGMA journal_mode=WAL` и `PRAGMA foreign_keys(1)` в `sqliteDSN()`; выполняет `migrationStatements` для создания таблиц.
- `Backend/internal/security/shield.go`: реализует `SecuritySystem`, выполняет `EdgeShieldMiddleware()` для HTTP calls и управляет `RunRetentionSweeper()`.
- `Backend/internal/security/audit.go`: через `RecordSecurityAudit()` вычисляет криптографическую цепочку `event_hash = SHA256(previous_hash || event_id)` и пишет в `security_audit_chain`.
- `Backend/auth/auth_service.go`: реализует `Authenticate()`, создает password hashes через `crypto.DerivePasswordHash()` и подписывает JWT через `jwt.SignToken()`.
- `Backend/messaging/messaging_service.go`: инкапсулирует `SendMessage()` и в `generateMessageProof()` создает SHA-256 digest ciphertext для хранения в `message_proofs`.
- `Backend/storage/storage_service.go`: координирует chunks в `SaveChunk()`, проверяет quotas через `CheckUserQuota()` и вызывает `RunFileRetentionSweeper()`.
- `Frontend/frontend/src/sovereignCrypto.js`: реализует `sovereignSeal()` и `sovereignOpen()`, проверяет hex-length через `assertHex()` и передает calls в Web Worker или Tauri bridge.
- `Frontend/frontend/src/sovereign/webCrypto.worker.js`: загружает `hqc-256.runtime.js` и WASM module `gaiacom_sovereign_wasm_bg.wasm` для изолированных key operations в Browser.
- `DesktopClient/src-tauri/src/sovereign_crypto.rs`: экспортирует пять native operations `sovereign_hqc256_keypair`, `sovereign_hqc256_encapsulate`, `sovereign_hqc256_decapsulate`, `sovereign_seal` и `sovereign_open` через `#[tauri::command]`.
- `Core/rust/gaiacore/src/lib.rs`: валидирует identities в `validate_gaia_id(value: &str)` по pattern `@user:domain.tld` и выполняет timing-resistant buffer comparisons в `constant_time_eq()`.

11. Архитектурные наблюдения и неиспользуемые файлы

Глубокий анализ codebase выявил следующие архитектурные особенности, исторические артефакты и redundancies.

1. Устаревшая реализация Vault (useVault.js против useDrive.jsx)

- **Наблюдение:** VaultPane.jsx существует в Frontend/frontend/src/components/chat/, а useVault.js - в hooks/.
- **Анализ:** в AppMainContent.jsx и App.jsx useVault полностью заменен на useDrive.jsx и DrivePane.jsx. useDrive.jsx прямо указывает это в header ("*Replaces useVault with GaiaDrive*"). VaultPane.jsx и useVault.js являются неиспользуемыми историческими артефактами.

2. Дублированные Federation Forwarding Routes

- **Наблюдение:** Backend/routes.go регистрирует два эквивалентных forward routes:

```
router.POST("/.well-known/gaiacom/s2s/v1/forward", fedHandler.HandleS2SForward)
router.POST("/_gaiacom/s2s/v1/forward", fedHandler.HandleS2SForward)
```

- **Анализ:** /_gaiacom/s2s/v1/forward сохраняется как backward-compatibility alias для старых clients, тогда как /.well-known/gaiacom/... соответствует standardized RFC-style discovery scheme.

3. Статический System User в Database Migrations

- **Наблюдение:** Backend/database/database.go создает hard-coded user 00000000-0000-0000-0000-000000000000 (system_remote_hub):

```
INSERT OR IGNORE INTO users (id, username, ...) VALUES ('00000000-0000-...', 'system_remote_hub', ...)
```

- **Анализ:** он служит synthetic foreign-key anchor для federation objects и S2S hub messages, позволяя выполнять foreign-key constraints (ON DELETE CASCADE) без реального user login.

4. Migrations: ALTER TABLE после первоначального объявления таблиц

- **Наблюдение:** в Backend/database/database.go несколько таблиц сначала создаются, а затем расширяются более поздними migrations через ALTER TABLE (например public_channel_posts ADD COLUMN pinned_at, public_channels ADD COLUMN comments_enabled).
- **Анализ:** это результат непрерывной эволюции программного обеспечения. Migration system применяет изменения инкрементально; для новых installations в долгосрочной перспективе рекомендуется consolidated baseline.

5. Граница миграции Dual-PQC (Legacy v0.1/v0.2 против Sovereign v1.0)

- **Наблюдение:** `Backend/messaging/messaging_service.go` и `Frontend/frontend/src/crypto.js` содержат parsing code для legacy algorithm suites (`GaiaCom/v0.1/hybrid-kem/...`).
- **Анализ:** это позволяет читать старые message packages в Inbox во время Beta v2, тогда как новые outgoing messages должны использовать Sovereign v1.0 (Accelerated / Top Secret).

6. Mobile Sovereign v1.0 Message Path как определенная Integration Boundary

- **Наблюдение:** Android-архитектура в `Android/` содержит полный module skeleton и встраивает Go node через `Backend/mobileapi`. Dual-PQC message path с native HQC-256 cascade продуктивно проверен для Browser и Desktop (Tauri); mobile end-to-end integration все еще находится в состоянии Technical Beta.

7. Административные PowerShell Scripts в Project Root

- **Наблюдение:** `scripts/` содержит helper scripts (`accept_android_licenses.ps1`, `build_android_native.ps1`) для CI/CD pipelines и developers; они не нужны во время runtime server operation.

Документация подготовлена и сверена с фактическим состоянием проекта GaiaCom. Статус: Sovereign Beta v2.